



YaIPS

Manual

Based on version V1.00 from August 21, 2026

Version history

22.06.2026 RR: First version.

Table of contents

1. Introduction.....	4
1.1 About YaIPS.....	4
1.2 What can YaIPS do?.....	4
1.3 About images and image channels.....	5
1.4 User Interface Overview.....	6
1.4.1 Main window.....	6
1.4.2 Dialogs.....	6
1.4.3 Tool Window.....	6
1.4.4 General information about windows and dialogs.....	7
2. The main window.....	8
2.1 Left area.....	8
2.1.1 Menu bar.....	8
2.1.2 Display settings.....	10
2.1.3 Information Area.....	11
2.2 Middle area.....	12
2.3 Right area.....	12
3. Dialogs.....	13
3.1 Settings.....	13
3.2 Calibration.....	13
3.3 Custom Colors.....	14
4. Tool window.....	15
4.1 Groups of Tool Windows.....	16
4.2 Source.....	17
4.2.1 Camera.....	17
Operating mode area camera.....	18
Operating mode line scan camera.....	19
Operating mode height camera.....	20
4.2.2 New Image.....	21
4.2.3 Image Viewer.....	21
4.2.4 Video Player.....	22
4.3 Processing.....	23
4.3.1 Filter.....	23
4.3.2 Color.....	25
4.3.3 Geometry.....	28
4.3.4 Calculation.....	29
4.3.5 Correlation.....	31
4.3.6 Objects.....	34
4.3.7 Other.....	36
4.4 Output.....	37
4.4.1 Creating a video.....	37
4.5 Inspection.....	38
4.5.1 Reference image.....	39
4.5.2 Color Testing.....	40
4.5.3 Position correction.....	41

4.5.4 Edges.....	42
4.5.5 Compare.....	43
4.6 Other.....	44
4.6.1 Overlay.....	44

1. Introduction

1.1 About YaIPS

YaIPS is an abbreviation for "Yet another Image Processing Software". Literally translated, this means "Yet another image processing software" and humorously alludes to the fact that there are many such programs.

Unlike **image editing** (which visually alters an image, e.g. retouching, color correction), **image processing** involves analyzing or automatically evaluating an image (e.g. measurement, object recognition). While the two terms sound similar, they pursue entirely different goals.

Industrial **image processing** uses cameras and software to **automatically** detect features, defects, or properties of products - usually directly on a production line. The goal is **to monitor** and **control manufacturing processes** or **to inspect finished products**.

For further information on **industrial image processing, lighting, optics, or cameras**, it's worth taking a look at the [Vision Doctor](#) website. These topics are explained clearly there in both German and English.

YaIPS is a program that allows you **to playfully explore the algorithms and methods used in image processing**.

Particular emphasis was placed on

- to make the results of each processing visible,
- to keep the operation simple and intuitive.

YaIPS is therefore ideally suited for **trying out** and **understanding image processing**.

I wish you much joy and many exciting insights while working with **YaIPS**.

1.2 What can YaIPS do?

- **Image analysis**
Color channel display, false color representation, 3D representation, measurement of color values, color histograms, horizontal or vertical brightness profiles, measuring distances.
- **Import/generate images**
from camera, file, video player, or generate images in defined sizes.
- **Image processing**
filtering images, modifying color, geometric corrections, image processing, correlation, object extraction, ...
- **Output images.**
Save images as .jpg, .png or .tga. Create videos.
- **Inspect images**
Take a reference image, measure or correct color, correct position, measure distances, compare image parts.
- **Other**
Overlay images, shapes or text (can also be used to create posters or shareable images).

1.3 About images and image channels

An image is a matrix of pixels, the total number of which is calculated as width × height. The pixel depth determines how many bytes are stored per pixel.

- **Width**

The width of an image indicates how many pixels are arranged horizontally next to each other. It therefore determines the number of pixels per line.

- **Height**

The height of an image describes how many pixels are stacked vertically on top of each other.

It indicates the number of pixels per column.

- **Pixel Depth**

Pixel depth indicates how many bits or bytes are stored per pixel. It determines how many different values a pixel can assume—that is, how many colors or shades of gray can be represented.

YaIPS uses images with 1 to 4 bytes per pixel. The bytes are also referred to as image channels. Each byte can have one of 256 possible values.

The alpha channel is an **additional channel** (transparency) that describes how transparent a pixel is.

The pixel depths used by YaIPS are:

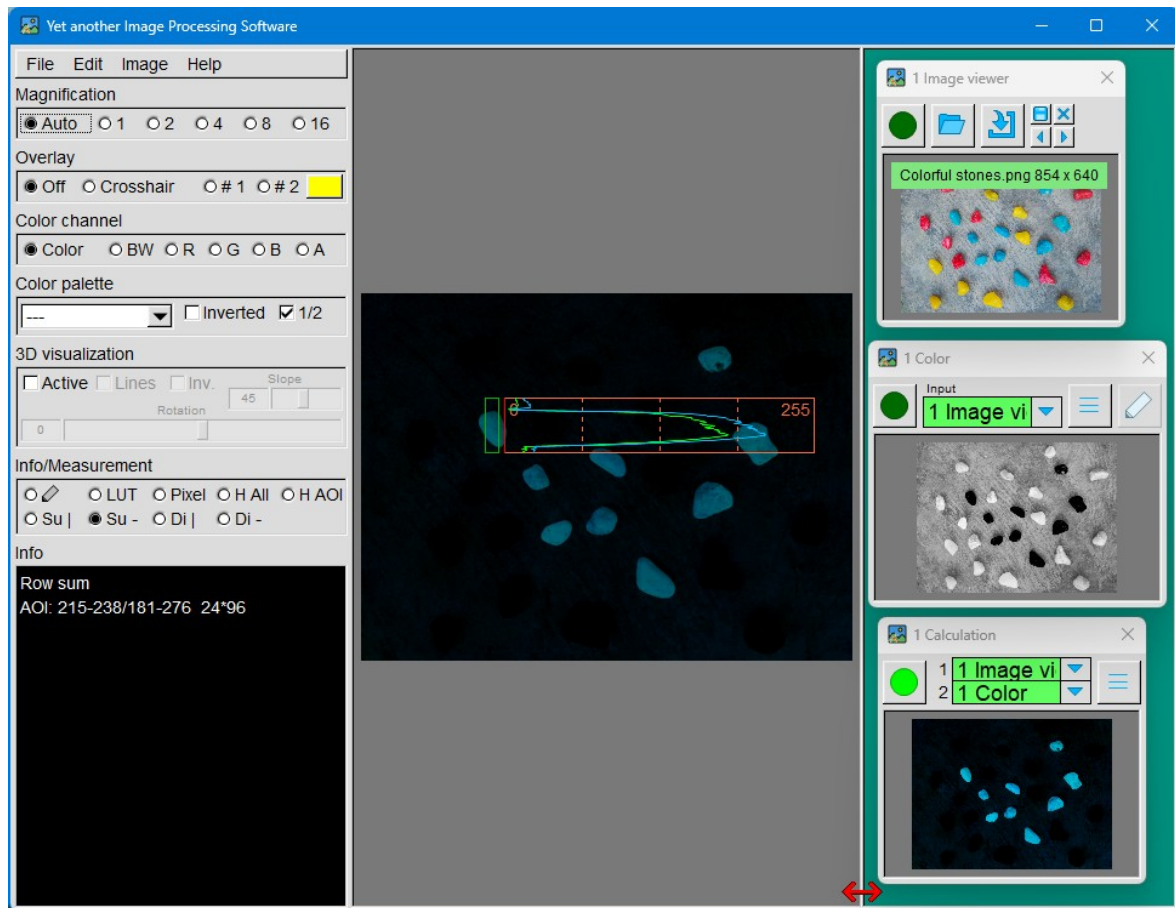
Number of bytes	Abbreviation	Remark
1	BW	Black-and-white, 256 shades of gray
2	BW+A	Black-and-white + 1 byte alpha (transparency)
3	RGB	Red, Green, Blue, 3×8 bits = 16.7 million colors
4	RGB+A	3 bytes color + 1 byte alpha (transparency)

Multiplying the width, height, and pixel depth gives the image size in bytes. These three parameters directly influence storage requirements, image quality, and processing effort.

Example calculation for a 1920 × 1080 RGB image:

$1920 \times 1080 \times 3 = 2,073,600$ bytes or 1.977 megabytes.

1.4 User Interface Overview



The user interface consists of several windows and dialogs that work together.

1.4.1 Main window

The **main window** is the central element of the user interface. It is divided into three areas:

- **Left area:** This contains the menu bar as well as numerous functions to display, change or analyze the image in the middle area.
- **Middle area:** The current image is displayed in this area.
- **Right area:** This area is used to place the **tool windows**.

1.4.2 Dialogs

There are various dialogs for settings, calibrations, or custom colors. These affect either the **main window** or individual **tool windows**.

1.4.3 Tool Window

Tool windows are responsible for the actual image processing.

- Each tool window can create or further process images.
- Tool windows can be linked together to perform multi-stage processing steps.
- The **output image** of a tool window can be displayed in the main window.
- Most tool windows have their own **parameter dialog**, through which the respective processing is set.

1.4.4 General information about windows and dialogs

- **Modal vs. Non-modal Dialogs**

A **modal dialog** must be **closed** before you can continue working in the application.

A **non-modal** dialog remains open without blocking the main window or other non-modal dialogs. You can therefore continue working while the dialog remains open.

In YaIPS, many non-modal dialogs can be open simultaneously, but only one modal dialog can be open at a time.

- **Keyboard focus**

At any given time, exactly one dialog box has keyboard focus. Keyboard input is therefore always directed to this window.

The focus dialog is indicated by the highlighted title bar.

- **Window positions and settings**

All windows remember their position on the screen and their settings.

These are automatically restored the next time the program is started.

- **Closing a Dialog**

All dialogs can be closed either by clicking the "X" in the upper right corner with the mouse or by pressing the Escape key.

- **Help text**

Help text is available for input fields, radio buttons and other GUI elements.

It appears automatically when the mouse remains briefly over the respective element.

- **Color Button**

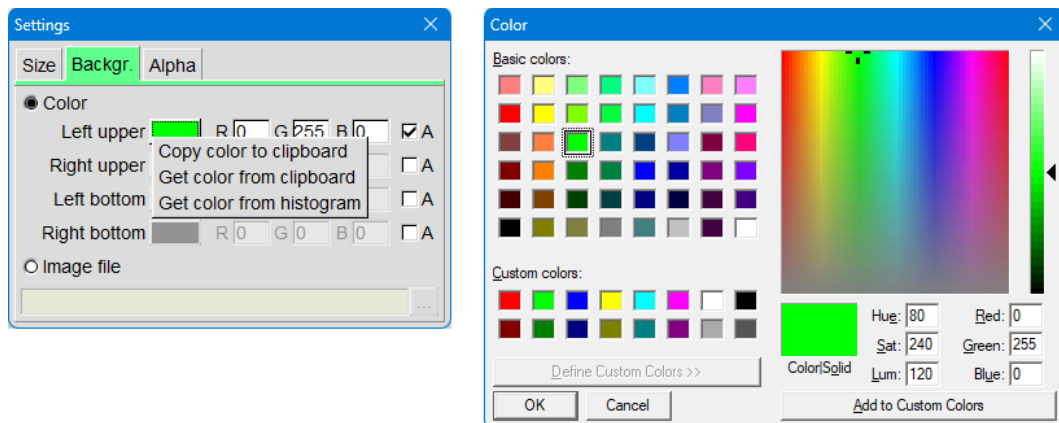
A **color button** is used to set a color. It is easily recognizable because the button itself is **colored**.

Right-clicking with the mouse opens the color selection dialog, where a new color can be chosen.

Left-clicking with the mouse opens a context menu. The current color can be changed there.

- to the **clipboard** or
- from the **clipboard**.

- If a **histogram is selected in the info** area of the **main dialog**, the color determined there can be applied directly as the button color.



2. The main window

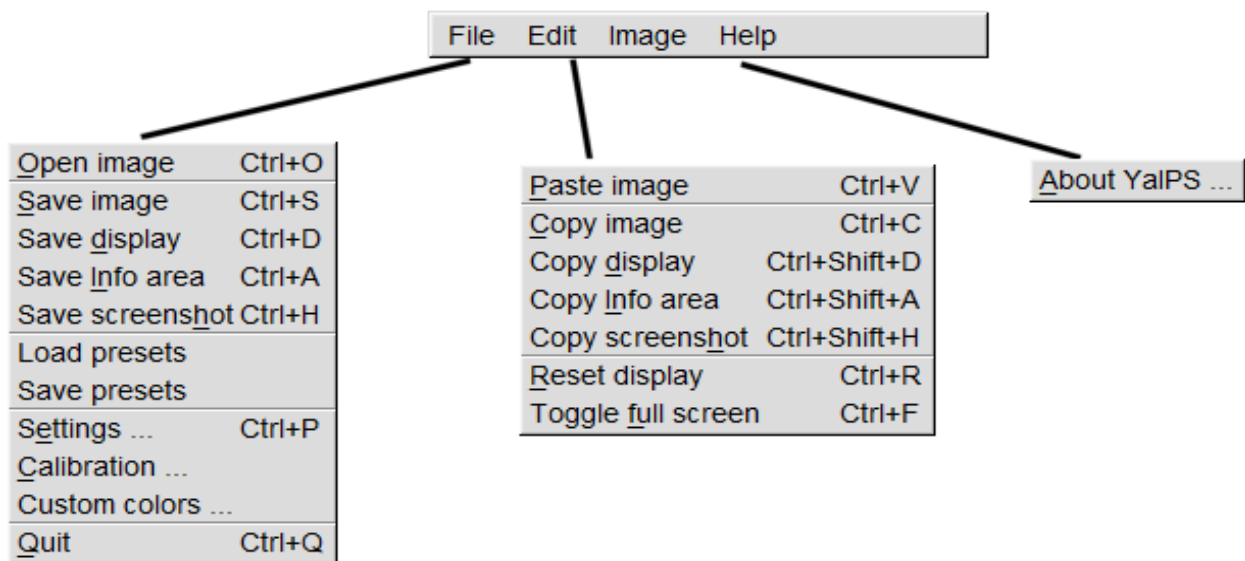
2.1 Left area

The **menu bar** is located at the top. This will be described in detail later.

Below are **settings** that directly affect the image in the middle section or extract information from this image. These functions will also be explained in detail later.

The **information area** displays various details about the currently shown image. Depending on the selected information or measurement function, **lookup-tables**, **brightness histograms**, or other image-related data may also appear here.

2.1.1 Menu bar



File

- **Open**

An image is loaded from a file and displayed in the middle area.

Supported formats: PNG, JPEG, JPG, TGA, BMP, GIF, PSD, HDR, PIC, PPM, PGM.

Alternatively, you can drag & drop an image file from Windows Explorer directly into the middle area.

- **Save**

The following content can be saved as an image:

- the currently displayed image in the middle area
- the entire middle area
- the information area
- a screenshot of the entire main window

Supported storage formats: **PNG, JPG, TGA** (PNG and TGA support images with **an alpha channel**).

- **Presets**

A preset (preset) is a file that saves **all current settings**.

- **Loading:**
When loading a preset, all currently open tool windows are closed first.
Then, the windows and settings saved in the preset are restored.
- **Save:**
All current settings of the main window and the tool windows are saved in a file with the extension ".prefs".
The state of the tool windows (open/closed) is also saved.

Note: Loading the preset "**ResetToDefaults.prefs**" will close all tool windows and reset all settings.

- **Dialogs**

The dialogs for settings, calibration, etc. will be described later.

- **Exit**

The application will be terminated.

Edit

- **Insert**

An image from the clipboard is loaded and displayed in the middle area.

- **Copying**

The following content will be saved to the clipboard:

- the currently displayed image in the middle area
- the entire middle area
- the information area
- a screenshot of the entire main window

Copying **does not** support an alpha channel.

Note: Pressing **Alt + Print Screen** saves a copy of a window - including the title bar and borders - to the clipboard.

- **Reset display**

All settings in the left area of the main window will be reset to their default values.

The image in the middle area will then be displayed **without any visualization changes** - exactly as it was originally loaded.

- **Switch to full screen mode**

This function toggles between **full-screen mode** and **windowed mode**.

The main window will then either fill the **entire screen** or be displayed as a **normal application window**.

Image

This submenu opens one of the tool windows. This will be described in detail later.

Help

Currently, only the "**About YaIPS**" dialog can be opened.

This window displays the **current program version** and the **open-source-projects** used.

2.1.2 Display settings

The screenshot shows a software interface for display settings. It is organized into several sections: 'Magnification' with radio buttons for Auto, 1, 2, 4, 8, and 16; 'Overlay' with radio buttons for Off, Crosshair, #1, and #2, and a yellow color swatch; 'Color channel' with radio buttons for Color, BW, R, G, B, and A; 'Color palette' with a dropdown menu, an 'Inverted' checkbox, and a '1/2' checkbox; and '3D visualization' with checkboxes for Active, Lines, and Inv., and sliders for Slope (set to 45) and Rotation (set to 0).

These settings directly affect the display of the image in the middle area.

Magnification

The magnification determines how large the image is displayed in the central area. With the **Auto setting**, the image size automatically adjusts to the available display area.

Overlay

Lines or a grid can be overlaid on the image.

The **color of the lines** can be freely adjusted.

Such guidelines are particularly useful when **setting up cameras** or **aligning objects**.

Color Channel

This setting determines which image channels are used for display:

- **Color:** All color channels of the image are used - the image appears in full color representation, or in the case of black-and-white images, only in black-and-white.
- **BW:** A color image is converted into a **black-and-white image**.
- **Individual color channels (R, G, B):** The currently selected channel is displayed as a **black-and-white image**.
- **A (Alpha channel):** If an alpha channel is present, it is displayed as a **black-and-white image**. If **no alpha channel** is present, the normal image is displayed instead.

Color Palette

Here you can select a **color palette** - also called a **Look-Up-Table (LUT)**. Each **input value of a color channel is converted into a new value using a table**.

A color palette consists of **three separate tables**. One each for the **red channel**, the **green channel**, and the **blue channel**.

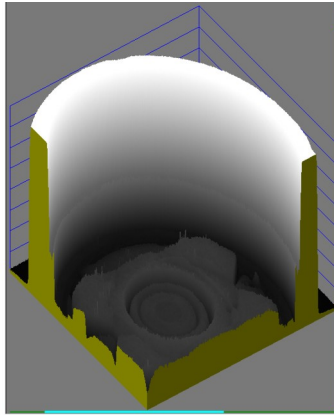
For black-and-white images: The image is colored by the LUT, as the tables generate new color values.

For color images: Each color channel is modified **individually** by its respective table. This means: Red is adjusted via the red table, green via the green table, and so on.

If **Invert** is enabled, the colors of the image in the middle area are reversed. Light areas become dark, and dark areas become light.

When **1/2 is enabled**, the brightness values of the image are halved. This makes the image appear darker, and overlaid graphics (e.g. lines, grids, AOI-markers) are more visible.

3D visualization

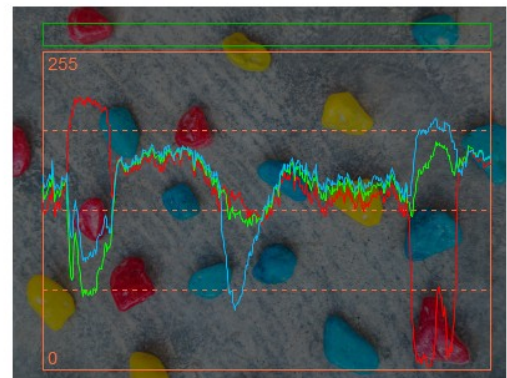
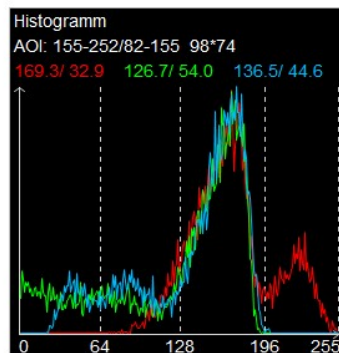
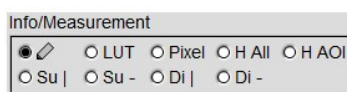


In 3D -visualization, the value of the selected color channel is interpreted as **height information** and displayed as a **height image**.

If the color channel **is** activated, a color image is automatically converted first into a black-and-white image to create a clear height reference.

The viewing angle and tilt of the 3D-view can be changed directly in the image by holding down the left mouse button and moving the mouse.

2.1.3 Information Area



Depending on the selected setting, various information can be displayed in the info area or on the image in the middle area, for example:

- the **current LUT**
- **Pixel values** under the mouse pointer
- **Brightness histograms**
- **Row -or column totals** within a rectangle
- **Distances** between points

For some of these functions, an **AOI-rectangle (Area of Interest)** is displayed in the image. This rectangle can be **moved** or **resized** with the mouse.

When the **pen icon** is activated, information about the current image is displayed, and the **learning mode** of certain tool windows is supported. Some tool windows themselves have a pen icon. In these windows, AOIs can be adjusted directly with the mouse while in learning mode. If such a tool window is active in the main window, AOIs can also be modified **directly in the large image area** - provided that:

- The corresponding option field is activated.
- and the pen icon is green

2.2 Middle area

This area displays images. These can appear in three ways:

- They are **loaded from a file**.
- They are inserted from the clipboard using **Ctrl + V**.
- An **image file** can be **dragged & dropped** directly from **Windows Explorer** into the middle area.
- They originate from a tool window whose **green dot** has been activated. In this case, the **output image** from that window is displayed here.

The **image size** can be changed in two ways:

- Via the **zoom options in the left panel**.
- By **rotating the mouse wheel** directly over the image.

If an image is enlarged, the visible area can be moved by **holding down the left mouse button** and moving the mouse.

For certain functions, an **AOI (Area of Interest)** appears in the middle area.

- This AOI can be **repositioned** and **resized** using the mouse.
- In the image at the beginning of chapter "1.4 User Interface Overview", a **row sum** is calculated from the AOI area (green rectangle), i.e., the average of all pixels in each row. The result is displayed **to the right of the AOI**.

2.3 Right area

Tool windows are located here. Each tool window can create or process images.

- The **output image** that a tool creates or modifies is displayed directly in the respective tool window.
- The **input images** required for processing are selected from the already open tool windows.

Clicking the **dark green dot** in the top left of the tool window displays the **output image** in the main window. The dot then turns **light green** to indicate that this image is active in the main window.

In the example at the beginning of chapter "1.4 Overview of the user interface", the following happens:

- In the **Image Viewer tool window**, an image is loaded from a file and displayed.
- The **Color tool window** extracts the **red color channel from this** and displays it as a separate image.
- The **Calculation tool window** then performs a **subtraction**: It subtracts the **output image of color** from the **output image of the image viewer**.

This subtraction makes the **blue stones** in the original image stand out particularly clearly.

Note: The boundary between the right and middle areas can be moved with the mouse (see red double arrow in the example).

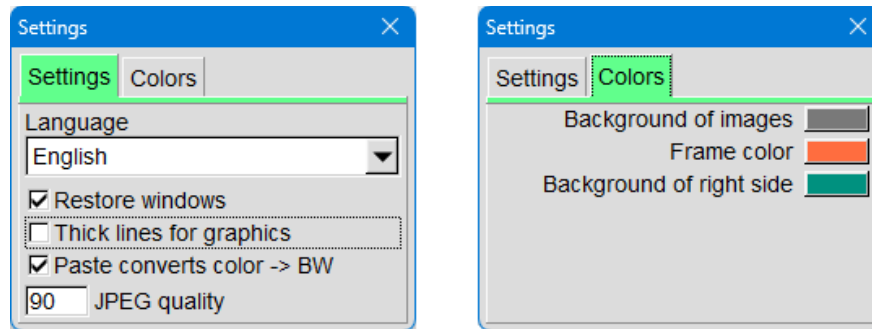
Note: The **tool windows** located **within the right area** are moved along when the main window is moved or when the boundary between the right and middle areas is moved.

Their position relative to the upper-left corner of the right area remains unchanged.

3. Dialogs

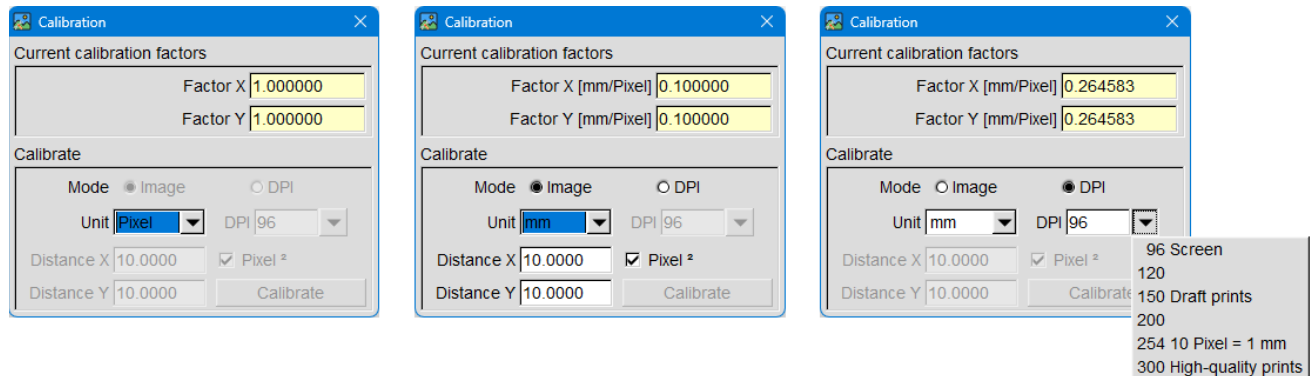
The dialogs described in this section are opened via the **"File"** menu bar. These dialogs affect either the main window or individual tool windows.

3.1 Settings



These settings primarily affect the **main window**. The **screenshots** usually provide a good overview. Additional information appears when the mouse **remains briefly over the respective interface element** – the corresponding **help text** is then displayed.

3.2 Calibration



This dialog manages the **calibration for the entire application**. It uses a base unit such as **pixels**, **mm**, **cm**, or **inches**. Many functions – such as distance measurements or size specifications – use this calibration to convert **pixel values into real-world units**.

Camera calibration

Calibration can be performed either with a **camera** or an **image**. For this to work, a **measuring object** (e.g. a ruler) must be visible in the image; a **known distance** is measured against this object.

- Positioning measuring objects
 - Click the green dot in the camera window to display the camera image in the main window.
 - Place the measuring device in front of the camera.
 - The distance to be measured should be as horizontal as possible.
 - An overlay can be activated in the main window for alignment purposes.
- In the calibration dialog
 - Select **"Image"** mode.
 - Specify the **unit** (mm, cm or inch).
 - Enter the known distance in **"Distance X"**.
- Perform measurement in the image
 - In the main window, under Info/Measurement, select **"Distance -"**.
 - Move the edges of the measurement window in the image so that they exactly cover the defined distance.
- Calculate calibration

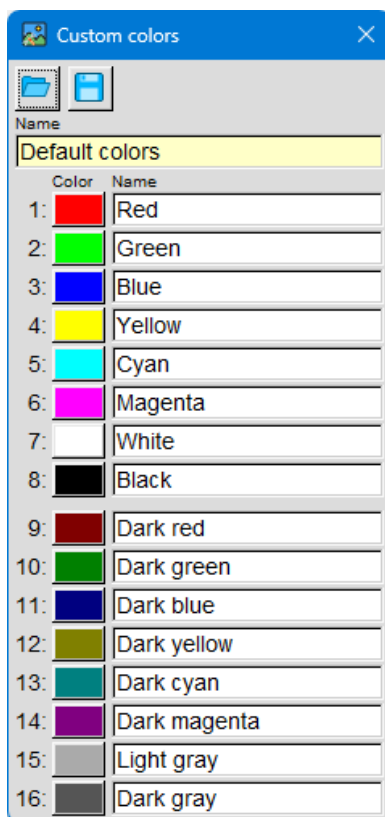
- In the calibration dialog, click on **"Calibrate"**.
- The calibration factors are now being calculated.
- For area scan cameras with **square pixels**, one measurement is sufficient.
- For **non-square pixels**, **line scan camera** simulation, or **height camera** simulation, both **horizontal** and **vertical** measurements are required.

Specify a resolution

This method is useful, for example, if you want to create images with a **defined size and resolution in the "New Image" tool window**.

- **calibration** dialog
 - Select the **"DPI"** mode (dots per inch).
 - Select a DPI preset or enter a value.
- In the **"New Image"** tool window
 - select a DIN size or enter the desired size manually.
 - Click **"Apply"**.

3.3 Custom Colors



This dialog allows you to create, save, and reload **color sets of up to 16 colors each**. You can also assign a **name** to each color.

The **last loaded color combination** is automatically used in the **color selection dialog of a color button**.

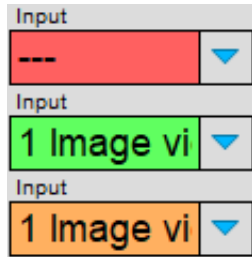
This allows, for example, a company's **corporate design colors** to be created and managed centrally.

4. Tool window

Tool windows are responsible for the actual image processing.

- Each tool window can create or process images.
- Many tool windows can be open in multiple instances simultaneously. They will then have a number displayed in the title bar.
- Tool windows can be linked together to perform multi-stage processing steps.

To link a tool window, select one of the **other open tool windows** as **input image** using the pull down button.

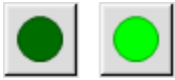


The status of the selected window is indicated by the **color of the name field**:

Red – No tool window has been selected yet.

Green – The selected tool window is **open** and has an **output image**.

Orange – The window is either **not open** or it is **not providing an output image**.

- The **output image** of a tool window can be displayed in the main window. Clicking the dark green dot in the upper left corner of the tool window  also displays the **output image** in the main window. The dot then turns light green to indicate that this image is active in the main window.

- Copy **output image** to the clipboard
Pressing **Ctrl + C** **saves a copy of the output image** to the clipboard.
- Most tool windows have their own **parameter dialog**, which is used to configure the respective processing settings.



If the tool window has keyboard focus, the parameter dialog can be opened using the **Ctrl + P** keys.

- **Pen-icon and learning mode**



Some tool windows have a **pen-icon**.

Clicking this button switches between **learning mode** and **evaluation mode**.

In **learning mode**, the pencil icon is **green**.

AOIs can now be adjusted directly in the tool window using the **mouse**. If the **source image is** displayed in the main window and the **pen-icon is also activated in the info area there**, the AOIs can also be changed directly in the main window using the mouse.

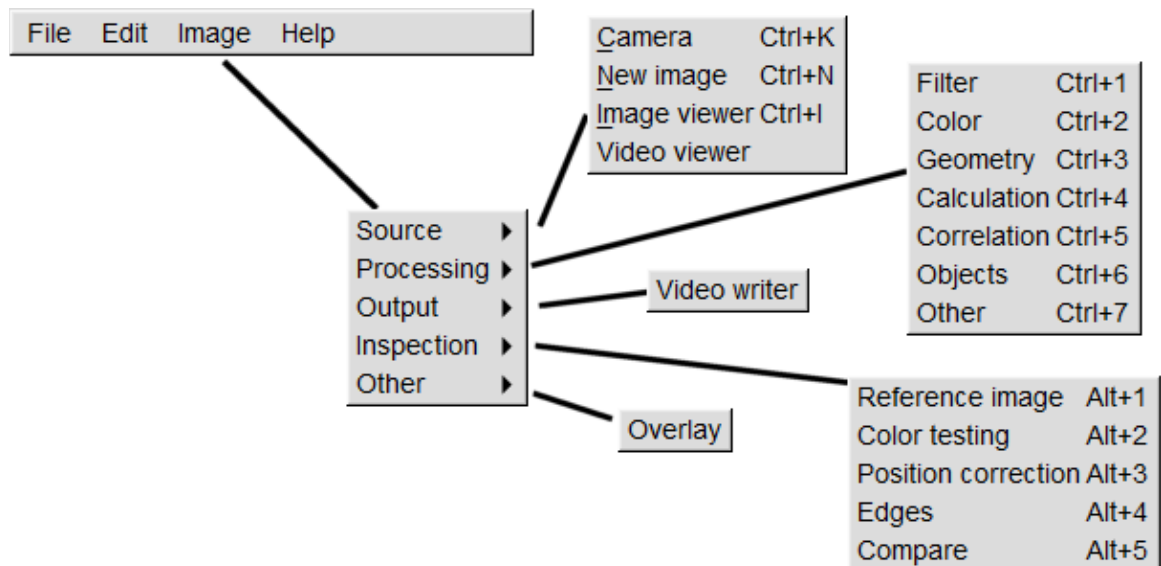
- Load from file



This button is used, for example, to load images from files.

- The **tool windows** located **within the right area** are moved along when the main window is moved or when the boundary between the right and middle areas is moved.
Their position relative to the upper-left corner of the right area remains unchanged.

4.1 Groups of Tool Windows



Tool windows are divided into different groups according to their common properties.

Here's an overview:

- **Source**
 - **Image sources** generate new images. They form the **starting point** of every **processing chain**. Images can be captured by **USB -cameras**, loaded from **files**, or **generated**, for example.
 - These tools **do not have an input image**.
- **Processing**
 - **Processing tools** manipulate one or more **input images**. Typical processing operations include: filtering, color correction, geometric adjustments, image processing...
 - The result is displayed as the **output image** in the window and can be used by other tool windows.
- **Output**
 - **Output tools** output an **input image**, for example, to a file or to a display.
 - They always have an **input image**.
 - Some generate an **output image**, but this cannot be further processed.
- **Inspection**
 - **Inspection tools** are used to analyze and compare images. Typical tasks include: capturing a reference image, measuring or correcting colors, correcting position, measuring distances or comparing image parts.
 - The most important element is the **reference image**, such as a camera image of an object. Other tools compare specific features of new images with the reference image to **detect or compensate for deviations**.
- **Other**

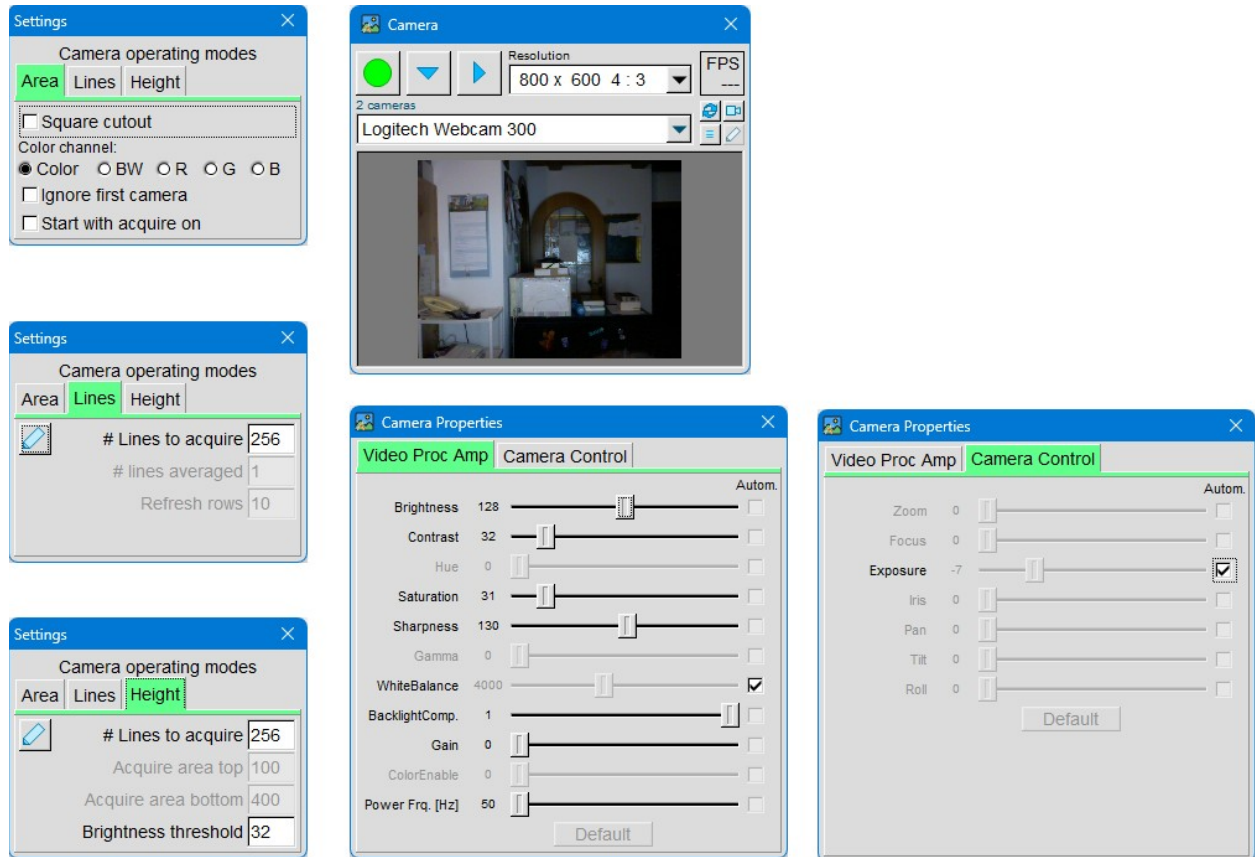
Other tools contain functions that do not fit into any of the tool groups mentioned so far.

4.2 Source

Image sources generate new images. They form the **starting point** of every **processing chain**. Images can be captured by **USB -cameras**, loaded from **files**, or **generated**, for example.

These tools **do not have an input image**.

4.2.1 Camera



Images can be captured by **webcams** and other cameras with **UVC** (USB Video Class) support.

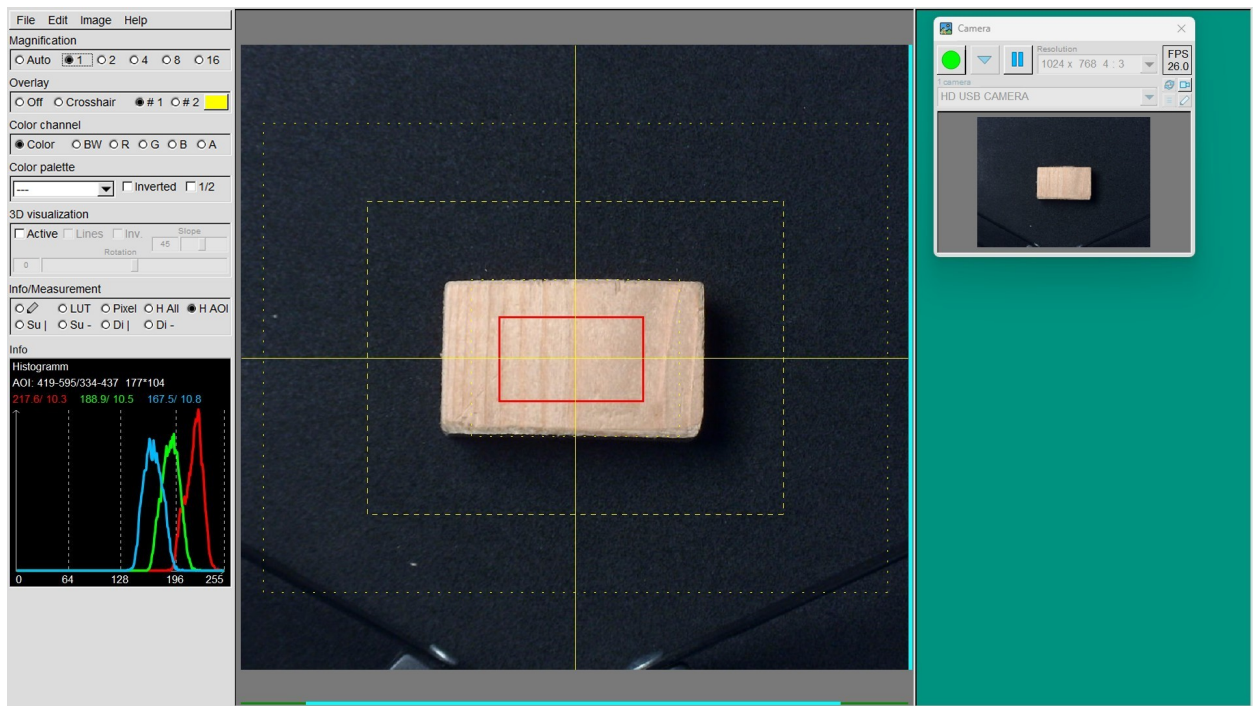
In the tool window, select one of the **cameras** and a **suitable resolution**.

In addition to **single-image capture**, **continuous recording** is also possible. If a camera is removed or reconnected, the available devices can be **re-searched**.

In the **parameter dialog**, select one of Three **operating modes**: area, lines or height (details follow later in the relevant section).

The **Camera Settings dialog** allows you to adjust various **camera parameters**, as the name suggests. These settings are **retained even after restarting the application**.

Operating mode area camera

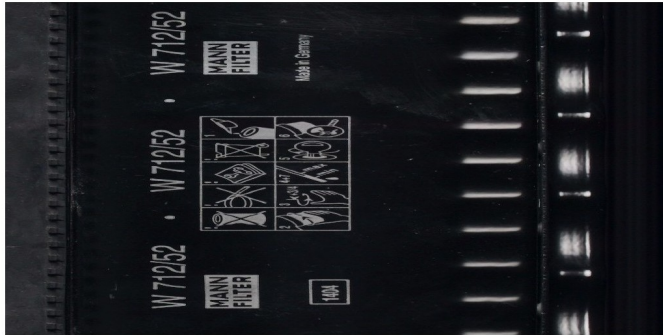
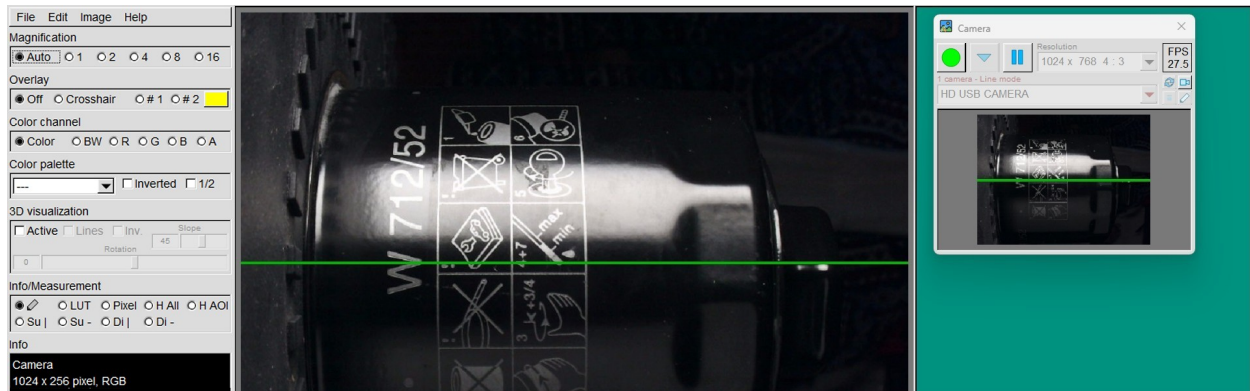


In **area scan mode**, standard camera images are captured. Almost all webcams are **area scan cameras**: The rectangular sensor delivers a **two-dimensional image** of the scene viewed through the lens.

A **normal webcam** is suitable for getting started. Even better is a camera with adjustable focal length and focus.

Using a tripod and appropriate lighting, stable and reproducible images can be produced, which are ideal for image processing experiments.

Operating mode line scan camera

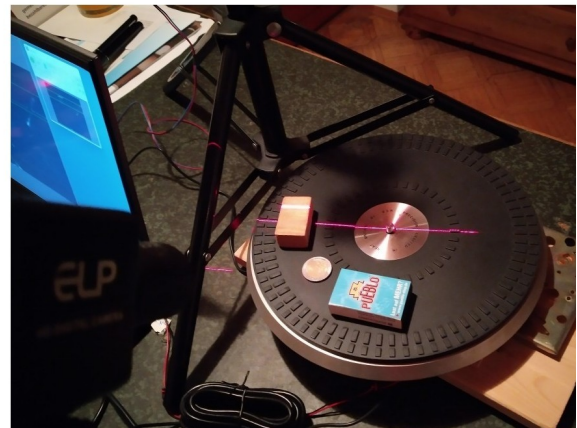
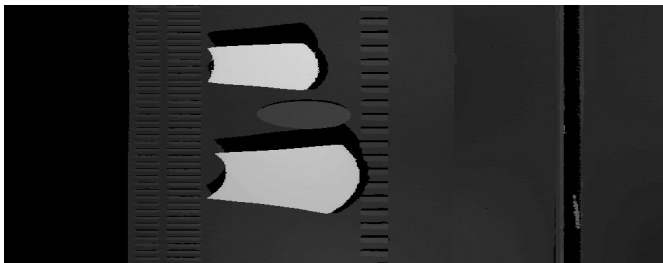
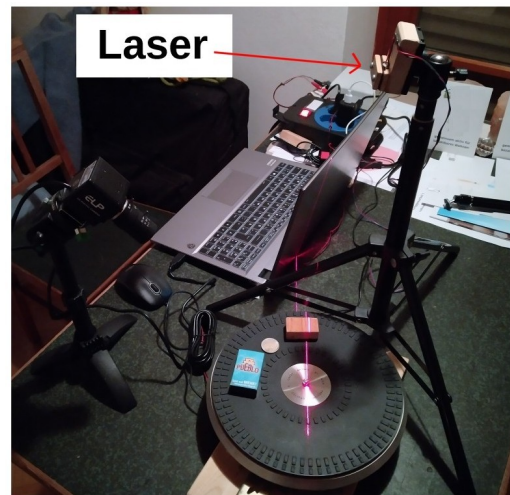
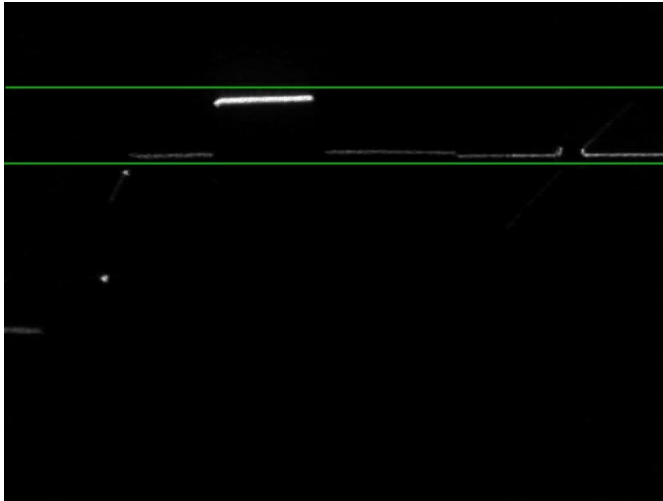


A **line scan camera** is a special type of camera that doesn't capture a **complete image at once, but rather a single line of pixels**. This line is captured very rapidly in succession as the object **moves in front of the camera** or as the camera moves past the object. A complete image is then created from many individual lines. It is used, for example, for scanning continuous materials or for unwinding round parts.

In **line scan mode**, YaIPS simulates a real **line scan camera** by using only a single pixel line from the image of an area scan camera. From many of these lines, a complete image is then assembled line by line.

The top image shows how the camera position and focus are set in the area scan camera's **learning mode**. The middle image shows a shot taken in **evaluation mode**. Here, the **unfolding of a round object is shown** - that is, the "unrolled" representation of the surface created by the rotation. The bottom images show the setup used. A turntable platter serves as a **rotating mechanism** that **spins smoothly** thanks to a small electric motor.

Operating mode height camera



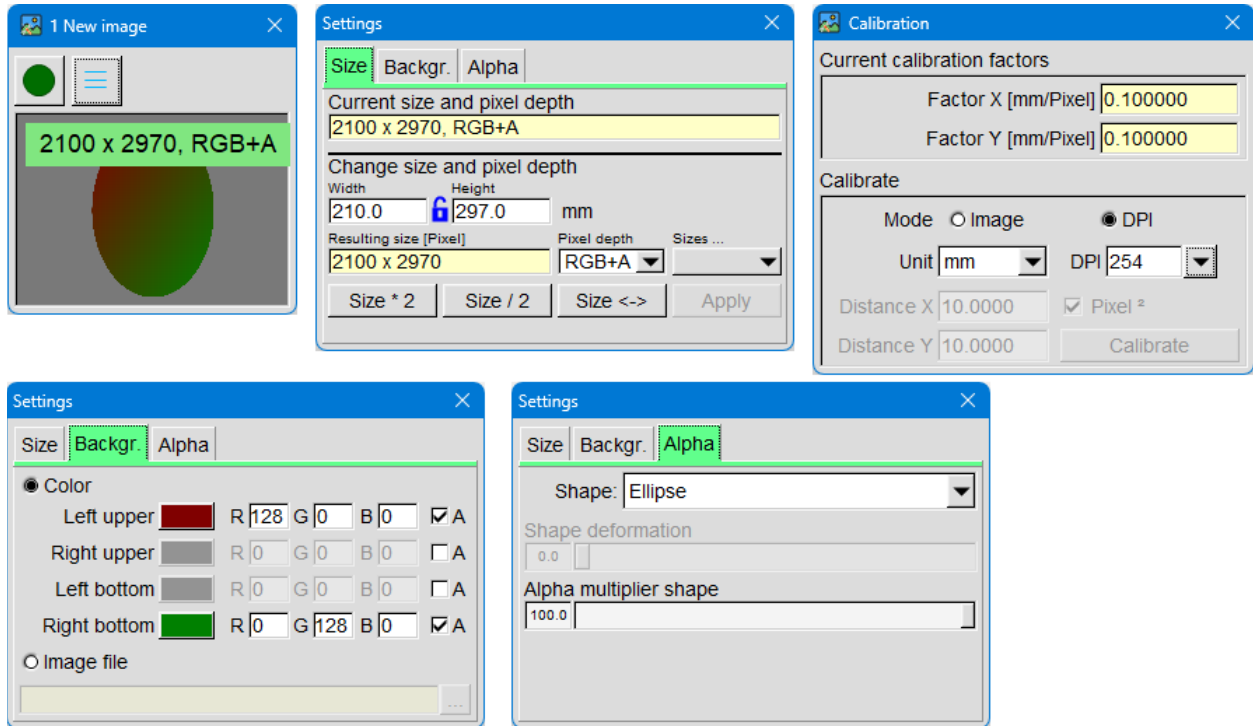
In **height camera mode**, YaIPS simulates a typical **laser triangulation measurement**. A **laser line** is projected onto the object, and an **area scan camera** captures the resulting **profile**. The object's shape distorts the laser line - this distortion is analyzed to obtain **height information**. To generate a complete height image, **many individual laser profiles must** be captured and combined into a single image. This requires either the **object to move** (e.g. rotate) or the **sensor** to be moved along the object.

The image in the upper left shows the camera setup in **learning mode**. The laser profile must be **horizontal** in the camera image. The **lower green line** marks the **lowest point**, and the **upper green line** marks the **highest point** of the object.

In **evaluation mode**, the **brightest point between the two lines** is searched for each image column. If it is closer to the lower line, the pixel is displayed **darker**. If it is closer to the upper line, the pixel is displayed **brighter**. This creates a **height image** in which the height of an object can be directly determined by its **brightness** (bottom left image).

The setup is shown on the right. A **turntable platter** serves as the rotating device and is turned smoothly by a small electric motor. The laser used is [PICO 70105094](#) **line laser module** (purchased from Reichelt Elektronik).

4.2.2 New Image



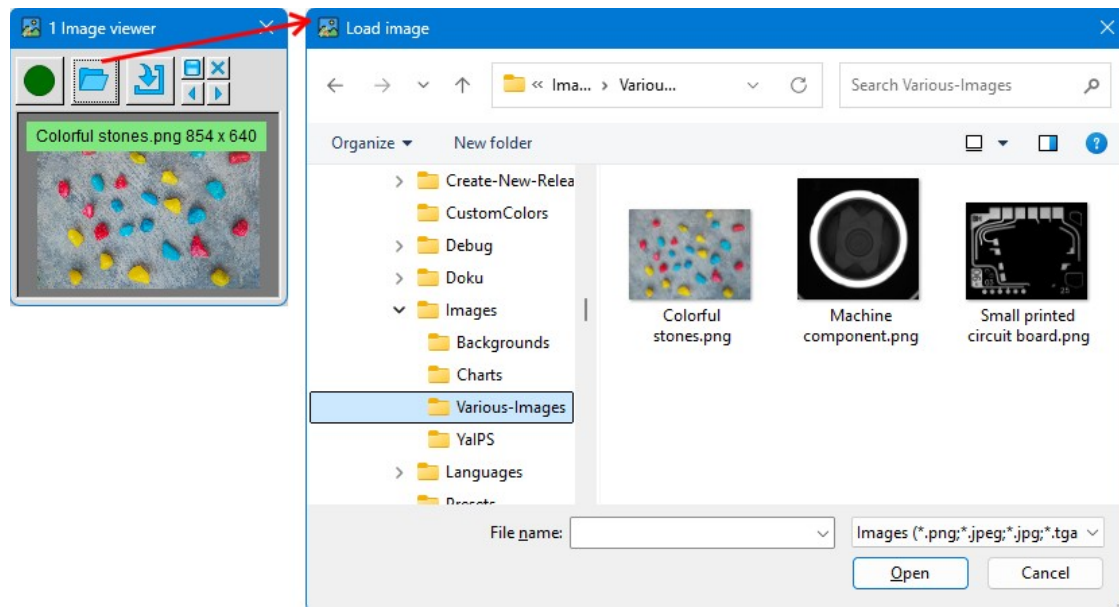
This tool window allows you to create **images of freely selectable sizes**. The program uses the unit of measurement and conversion factors from the calibration dialog to correctly convert the desired size into **pixels**.

After entering the desired image size and pixel depth, a new image is created by clicking "**Apply**". The size is automatically **converted to pixels**. Instead of manual input, a **predefined size can also** be selected.

The generated image can be filled with a single color, a gradient, or an image from a file.

If an image with an alpha channel is created, an **alpha mask can also** be defined.

4.2.3 Image Viewer

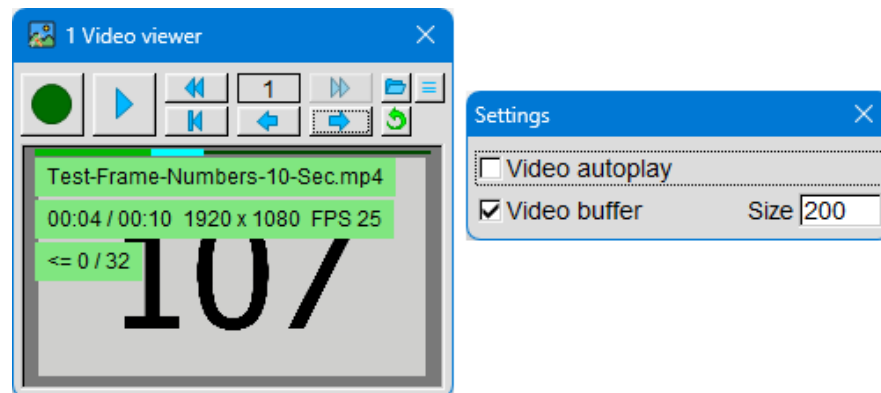


The **image viewer** loads images from the **hard drive** and displays them in the tool window. Alternatively, you can drag & drop an image file from Windows

Explorer directly into the image area or paste an image from the clipboard using Ctrl+V.

You can easily switch between the **next** and **previous image** in the file directory using the arrow keys. Additional buttons are available to move the currently displayed image to the middle area of the **main window**, **save** the **output image** to the hard drive, or **delete** it completely.

4.2.4 Video Player



The video player, as the name suggests, plays **videos**. The last opened file is automatically reloaded the next time the window is opened. If the corresponding option is enabled in the settings, **playback begins immediately after loading**.

Videos are loaded using the "Load" button or via **drag & drop** using **Windows Explorer**.

The **double arrow keys** change the **playback speed**. Use the **right/left arrow keys** to skip forward or backward in the video: During playback, the skip is **10 seconds**; during **pause**, it skips **one frame** at a time. If the **video buffer** is enabled, you can also rewind one frame. If the **Ctrl key** is pressed at the same time, the video skips forward **1 minute**; if the **Shift key** is pressed, it skips forward **10 minutes**.

A **progress bar** is located at the top of the screen. When the video buffer is active, the already played portion is displayed **in green**. With the video buffer enabled, the end of the played portion is highlighted **in light blue**. **These frames are cached and can be played backward step by step** during the pause.

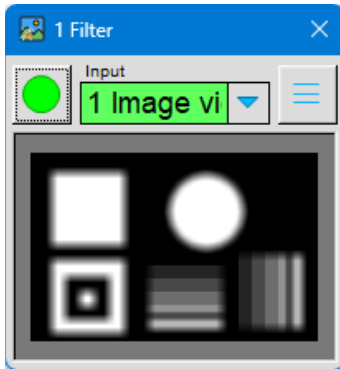
The **information in the image area** displays the **file name**, the **total length of the video**, the **elapsed time**, the **image size in pixels**, and the **frame rate**. If the video buffer is enabled, an additional line appears indicating the **buffer size in frames** and the **number of frames rewound**.

4.3 Processing

Processing tools process one or more **input images**. Typical processing operations include: filtering, color correction, geometric adjustments, image processing...

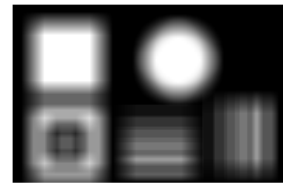
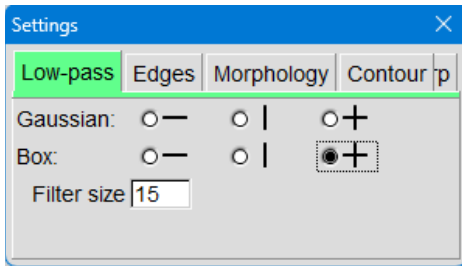
The result is displayed as the **output image** in the window and can be used by other tool windows.

4.3.1 Filter

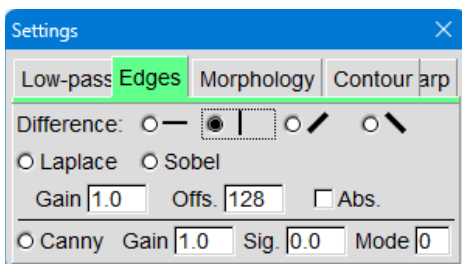


In image processing, a **filter is a computational method used to selectively modify** an image in order to highlight specific information or reduce noise. You can think of a filter as a **tool** that "edits" an image, similar to a tool in photo editing - only mathematically and automatically.

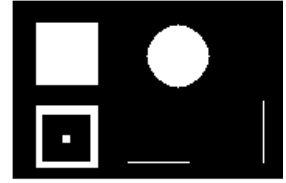
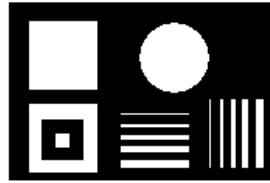
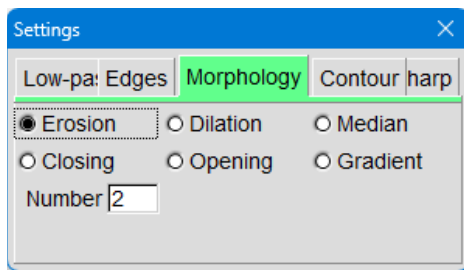
A filter changes the pixel values of an image. This usually involves examining the pixels surrounding a point and calculating a new value from them. In this way, certain characteristics of the image can be enhanced or reduced.



A **low-pass filter**, also known as a **smoothing filter**, makes an image appear smoother by reducing minor disturbances. Typical examples are the **average value filter** and the **Gaussian filter**. They blend the pixels of a given area, causing the image to appear blurred.



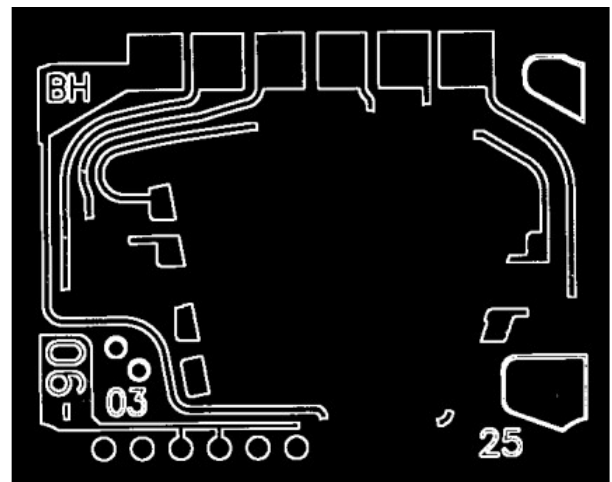
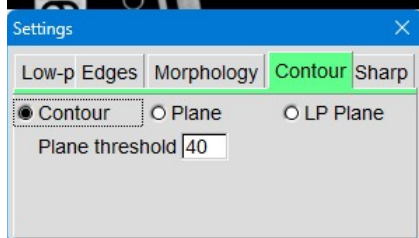
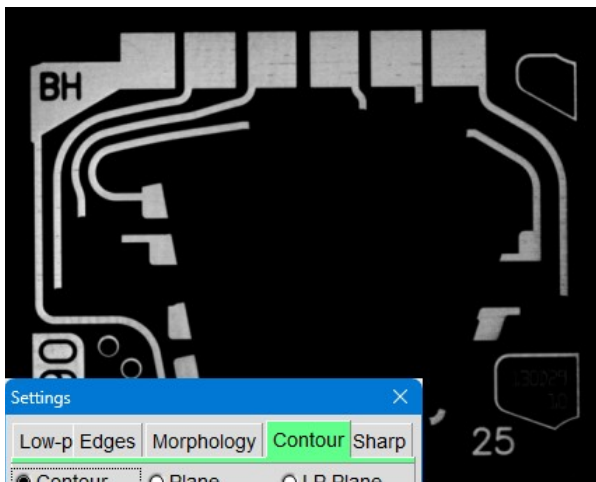
An **edge filter** highlights transitions between light and dark. Well-known variations include differential filters, **Sobel filters**, and **Canny filters**. They show where strong changes in brightness occur in the image - ideal for edge detection or object recognition.



Morphological filters are **image operations** that modify the shape of objects in an image by selectively adding or removing pixels. The most important basic operations are **erosion** (making objects smaller) and **dilation** (making objects larger). Combining these steps creates **opening** and **closing operations**, which, for example, remove small imperfections or fill gaps.

In **morphological operations**, a pixel is always **considered together with its immediate neighbors**.

In **erosion**, the **darkest pixel** in this neighborhood is the result. In **dilation**, the **brightest pixel** is used. With the **median filter**, the pixel whose brightness lies **in the middle** of all neighboring pixels is obtained.

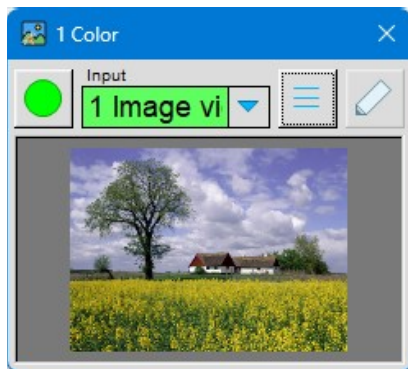


A **contour filter** checks how much a pixel differs from its immediate neighbors. If the **brightness difference** to at least one neighboring pixel is **greater than the level -threshold**, this pixel is **classified as a contour**; if the difference is smaller, it is considered a **plane**. Such filters are also suitable for **tolerant image comparisons**, where small differences are ignored, but significant deviations are reliably detected.



A **sharpening filter** enhances details by emphasizing edges. The image appears clearer because differences between pixels are more pronounced.

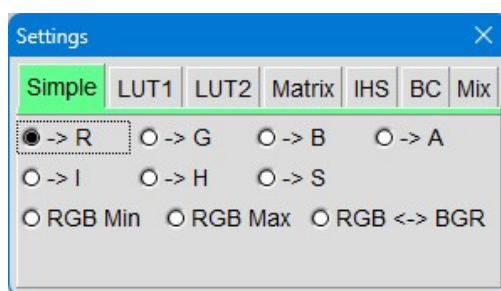
4.3.2 Color



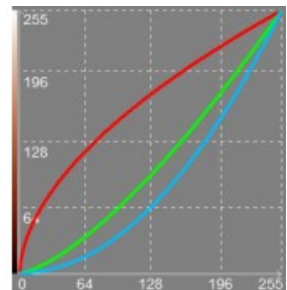
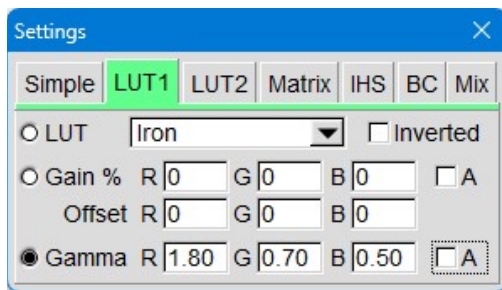
Color filters modify the **color values of a pixel** to selectively adjust or analyze images. **Lookup tables (LUTs)** are frequently used for this purpose, allowing for very fast and efficient conversion of color channels. **Matrix operations** can be used to combine the individual color channels of a pixel, for example, to change hues or emphasize specific color ranges.

Furthermore, color images can be converted to **black-and-white images** or back again, and **IHS color space** transformations can be performed, in which intensity, hue and saturation are processed separately.

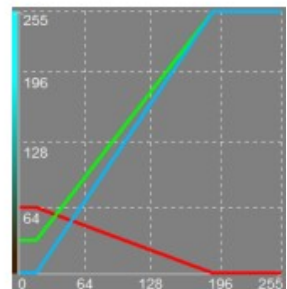
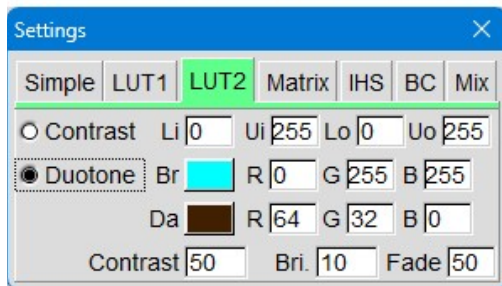
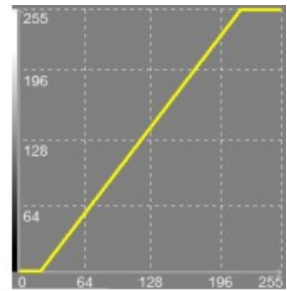
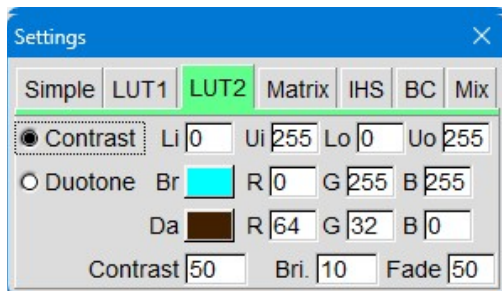
Color filters are used to **emphasize or suppress certain colors** so that objects are easier to recognize and interference is reduced by -considering only relevant color or contrast ranges.



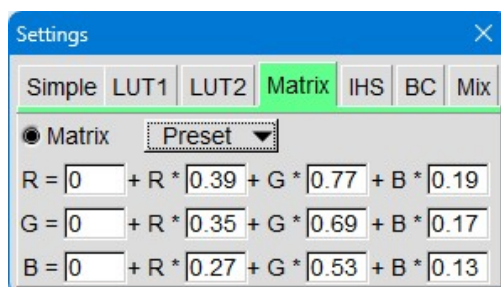
This tool extracts **individual color channels** from a color image, for example, **red, green, blue**, or the components of an **IHS conversion** (intensity, hue, saturation). Additionally, the **minimum and maximum values** of the RGB pixels can be determined. An **RGB to BGR conversion** is also possible, in which the order of the color channels is reversed.



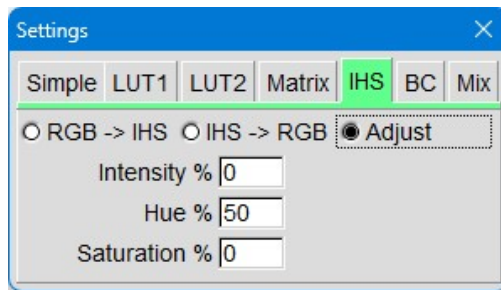
With this tool, you can select a **predefined lookup -table (LUT)**, as described under *Color Palette* in section 2.1.2. Additionally, you can **lighten or darken the colors** and apply **gamma correction**. Provided the [output image](#) is large enough, the resulting lookup table is **displayed as a graphic**, allowing you to see the effect of the color transformation immediately.



The **"Contrast"** setting allows you to define contrast ranges by specifying which brightness or color values should be considered light or dark areas. The **"Duotone"** effect filter converts an image so that it consists of only two colors: a dark one and a light one. The dark color is used for shadows, and the light color for highlights. All intermediate values are simply blended between these two colors. This creates a clean, graphic two-color look reminiscent of posters or screen prints. For color images, the conversion of the individual RGB channels can be combined with the conversion of a black-and-white image.

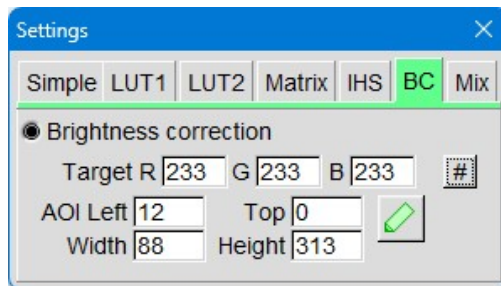


Color matrices are used **to selectively alter the colors of images**, for example, to give them a specific tint, style, or duotone look. Each pixel of the image is mathematically recalculated: The original RGB-values of a pixel are multiplied by a **3×3-matrix** that determines how strongly red, green, and blue are mixed.



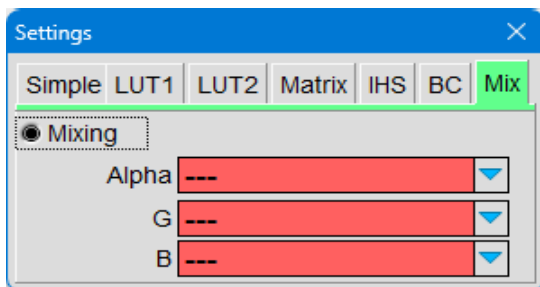
IHS is an **alternative color space** in which pixel values have a different meaning than in the standard RGB model. IHS breaks down an image into **intensity** (pixel brightness), **hue** (the specific color), and **saturation** (how saturated the color is).

Here, you can convert between RGB and IHS or selectively modify individual components.



Brightness **correction** of an image is useful when the lighting changes over time - for example, due to the aging of a light source or fluctuating environmental conditions. To do this, a **measurement window** is placed in the image on a spot that is clearly visible in every captured image and always in the same position. This area should be as bright as possible so that changes can be reliably detected.

For all subsequent captures, the brightness is adjusted so that the measured value in the measurement window corresponds to the desired target value. In this way, the image brightness remains constant over many captures, even if the actual lighting changes.



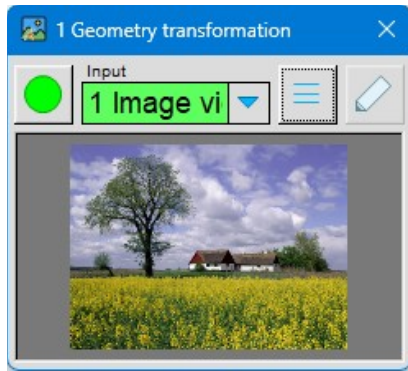
Mixing color channels to create a new image.

- * No image selected
An alpha channel is removed from the input image
- * G and B selected
Input, G, and B must be black/white images. A color image is generated (input is R).
- * Alpha selected
Output image has an alpha channel. If available, the alpha channel is used. Otherwise, alpha must be a black/white image.

This tool lets you edit **the alpha channel** of an image. You can **remove an existing alpha channel**, **add a new one**, or **replace** an existing one. This allows you to precisely control which areas of the image should be transparent or opaque.

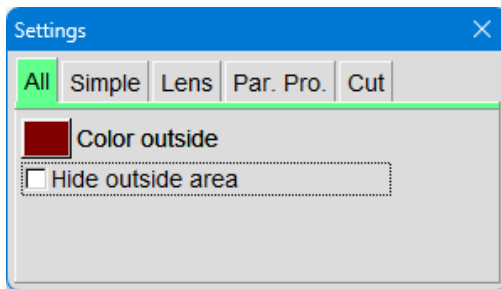
Additionally, the tool offers the option of combining **multiple black-and-white images into a single color image**. Each black-and-white image is used as a separate color channel - for example, one for **red**, one for **green**, and one for **blue**. In this way, a complete RGB image is created from separate channels.

4.3.3 Geometry



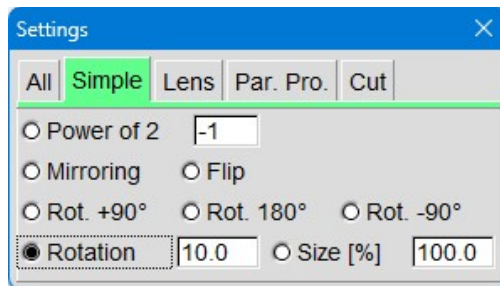
Geometry here refers to all **functions that change the shape or orientation of an image**.

In this tool window, you can **scale the image, rotate, flip, or move it**. More complex adjustments, such as lens correction (e.g. to remove distortion), are also possible. Additionally, you can select a specific **area of the image** and extract it as a separate sub-image.



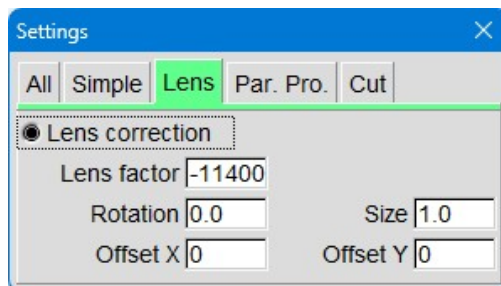
These parameters are used when an image **is rotated, distorted, or otherwise geometrically transformed**. This inevitably creates areas that lie **outside the original image**. Any **replacement color** can be assigned to these areas to prevent "empty" or undefined pixels.

Alternatively, this outer area can also **be hidden using an alpha mask**. In this case, the newly created image areas are displayed transparently, so that they can later be easily overlaid on other images or further processed.



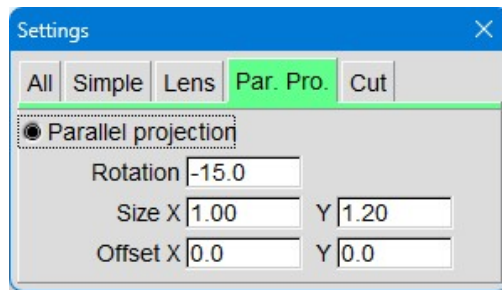
These basic geometric functions allow you to **easily resize or rotate an image**.

These operations are fundamental geometric editing techniques and are frequently used to align images or adapt them to a desired format.

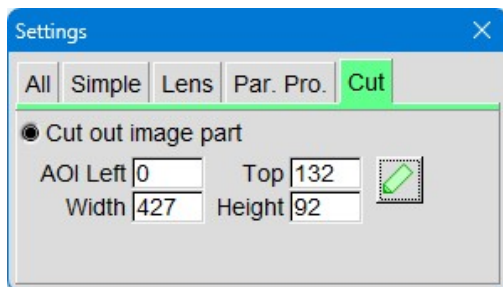


The **lens factor** corrects an optical distortion known as **pincushion distortion**. This distortion is often caused by certain lenses and results in the image appearing "stretched" towards the edges. By adjusting this factor, this distortion can be compensated for, restoring the image to a natural geometry.

In addition, further corrections can be made using the same parameters: The image **can be resized**, the **rotation corrected**, or a **lateral offset** compensated. This allows for the reliable correction of typical imaging errors caused by the camera lens or positioning.

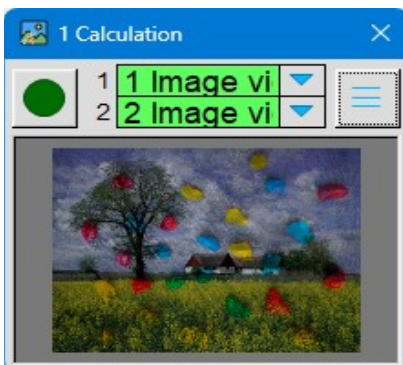


Parallel **projection** is another method of geometric manipulation. In addition to rotation or translation operations, you can adjust the image size separately for the X and Y directions.



In **learning mode**, an **area within the image** is defined. In **evaluation mode**, this previously defined area is **extracted** from the image and used as **the output image**.

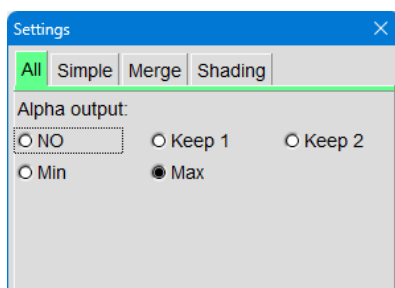
4.3.4 Calculation



The **calculation** combines functions that **process two image sources**. The pixel values of the two images can be directly combined - for example, by **adding, subtracting, or multiplying**. The **output image** always adopts the **size of the first input image**. All subsequent **input images** are automatically adjusted.

Additionally, various methods are available to **blend the two images together**. These blending methods determine how strongly each of the two images is visible in the final result.

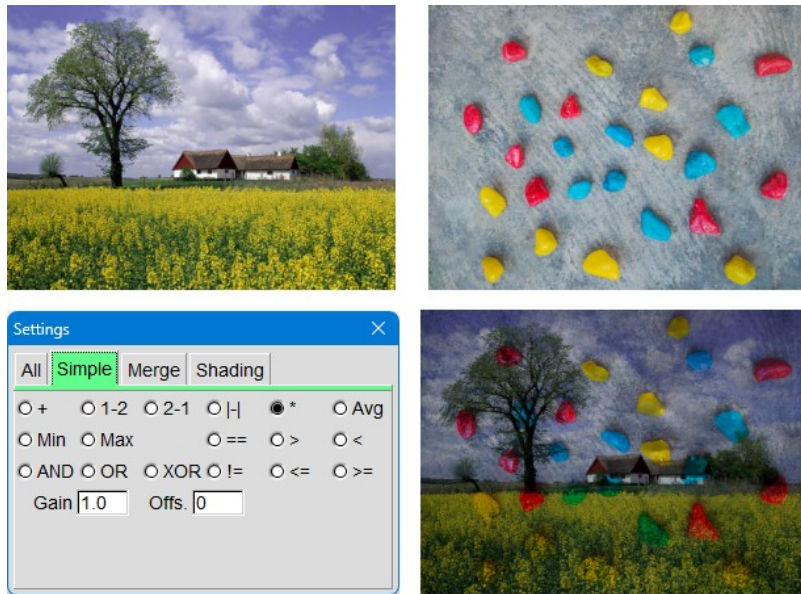
One special mode is **shading**. This allows you to balance differences in **shading or brightness** within an image. This is particularly helpful if an image is unevenly lit or if certain areas appear too dark or too bright. Shading ensures that the entire image has a more uniform appearance.



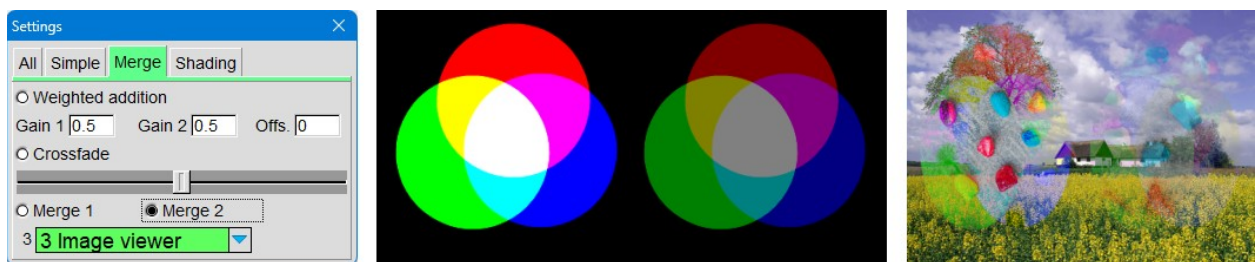
If **neither** image has an alpha channel, then the **output image also lacks an alpha channel** - it remains completely opaque.

If **only one** of the **input images** has an alpha channel, this can be **applied to the output image**. If **both input images** have an alpha channel, a **minimum or maximum calculation** can be performed. In this process, either the **smaller** or the **larger** alpha value from the two images is selected for

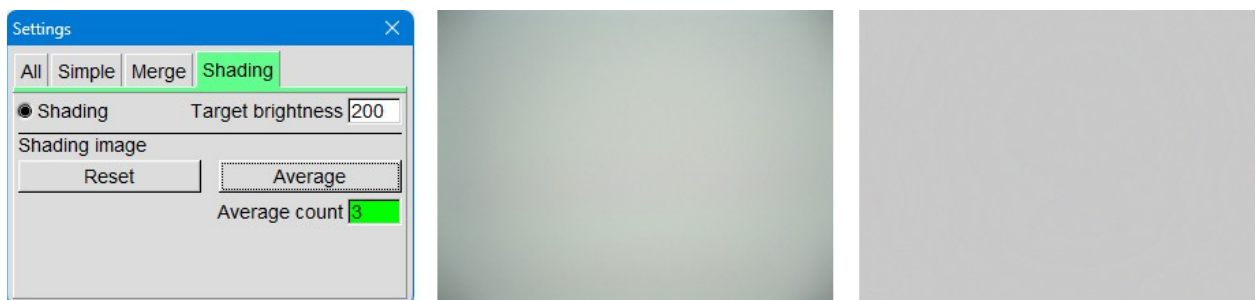
each pixel and applied to the **output image**.



In this example, the two images above are **multiplied together** so that their pixel values are combined to create a new **output image**. After each calculation, the **output image** can be further adjusted: The **gain setting** allows you to selectively **lighten or darken the result**, while the **offset** adds a fixed brightness value to shift the overall image to be brighter or darker.



Here, the **first two input images** are **blended together**. How strongly each of the two images is visible in the result depends on the **third image** (the middle image). The **brightness** of this middle image determines the **blending ratio**. **Bright** areas in the third image cause more of the **second image** to be visible. **Dark** areas cause more of the **first image** to be visible.



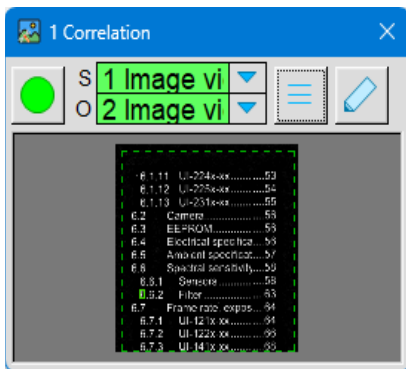
The middle image shows a *camera snapshot* of a **white surface**. **Darkening** is visible in the corners - typical effects caused by the lens or uneven lighting.

The shading -function is used to compensate for such differences in brightness, so that the image later appears uniformly bright everywhere.

Procedure for using the shading -function:

- The first **input image 1** is the **camera image**.
To set up the shading, a **white sheet of paper** is placed under the camera.
- **Input image 2** must be an **image viewer**.
It is used to temporarily store the so-called **shading image**.
- Pressing **Reset** deletes the current shading -image.
Pressing the **Average** button for the **first time** saves **input image 1** as the new **shading image**.
- To compensate for minor irregularities in the paper, the paper can be **repositioned**.
By pressing **Average** again, the new camera image is **averaged** with the previous shading image. This process can be **repeated several times** - typically **4 to 8 times**.
- The **output image** is finally a **shading corrected camera image**.
If the paper is removed and a bright object is held in front of the camera, the **corners** will be just as bright as the **center** of the **image**.

4.3.5 Correlation



In image processing, **correlation** means **comparing** two images or two patterns to find out **how similar** they are at a specific point.

Correlation checks: *"Does this small pattern (object) fit in this place in the image (scene)?"*

The pattern is moved across the image, and a value is calculated at each position indicating **how well the two elements match**.

High similarity → high correlation value

Low similarity → low correlation value.

The **cross-difference correlation** used here is a really simple type of this method: It measures **how much two image regions differ** from each other.

This is done by calculating the **difference in pixel values** between the object and the scene.

For each pixel: $\text{Difference} = |\text{Pixel Scene} - \text{Pixel Object}|$

Then all the differences are **added up**.

- **Small sum** → Image and pattern are **similar**
- **Large sum** → Image and pattern are **different**

The less the pixels differ, the better the pattern fits in that position.

Cross-difference correlation is fast, simple, and works well when the **lighting is stable**.

Note: Correlation can be **very computationally intensive** for **large scenes** or **very large object images** because a large number of pixels need to be compared.

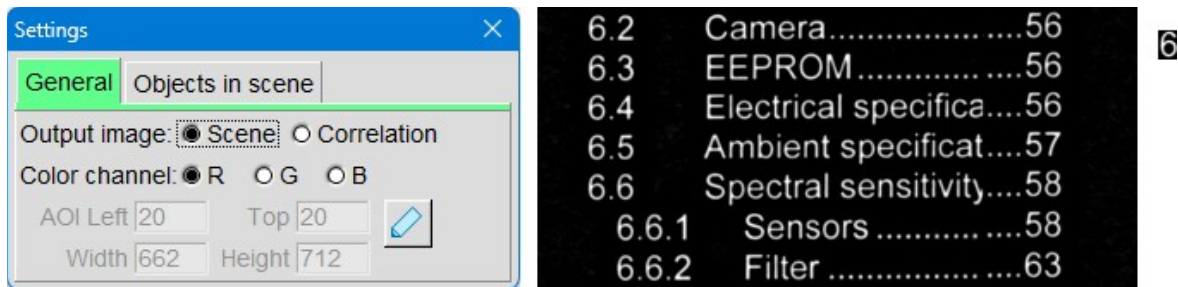
Therefore, it is often advisable **to reduce the image size before correlation**. This reduces the number of pixels that need to be compared, and the calculation runs **significantly faster** without severely impacting the result.

The images for the following correlation examples are located in the "Demo\Correlation" subdirectory of the application.

The **raw data of the scene** - a labeled sheet of paper - was previously prepared by **background elimination**. This involves applying a **small Gaussian filter** and a **large Gaussian filter** and subtracting their results from each other, thus largely removing the background.

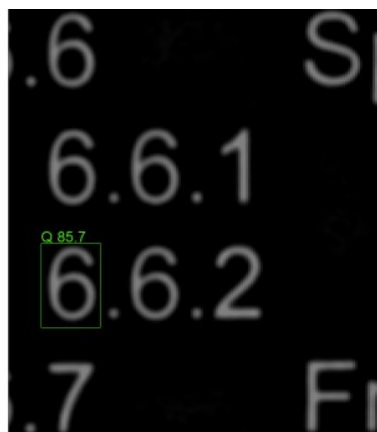
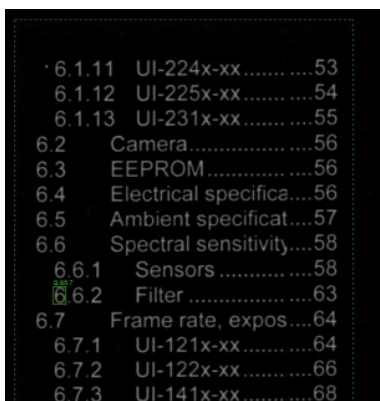
objects were then cut out from this cleaned-up scene image and saved as **small individual images**.

Note: Objects can be quickly replaced by **drag & drop** them from the File Manager directly onto the **image viewer for objects**.

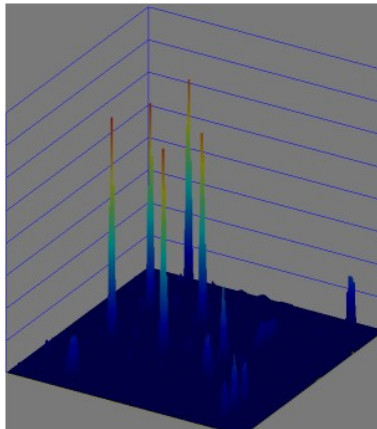
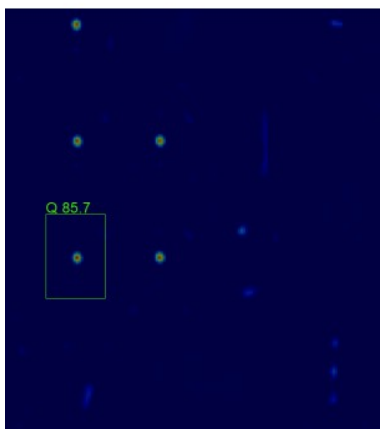


The **middle image** shows a **section of the scene**, the **right image** shows the corresponding **object**.

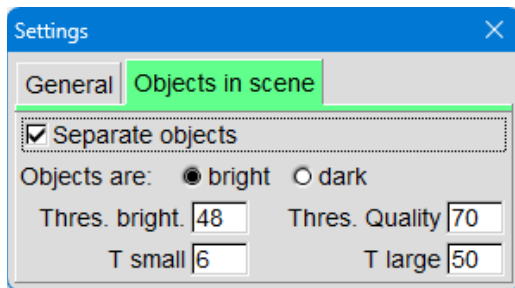
The **object image** must **always be black-and-white** for the correlation to work correctly. The **scene image**, on the other hand, can be **black-and-white or color**. If it is a **color image**, only the **selected color channel** will be used for the correlation.



If "Scene" is selected as the **output image**, the scene is displayed **along with the superimposed result of the correlation**. The **rectangle** marks the **position of the best match**. The **correlation value** ranges from 0 (no match) to 100 (optimal match).



If "Correlation" is selected as the **output image**, a **processed image of the correlation qualities** appears. Areas with **good agreement** are **light**, areas with poor agreement **are dark**. This is particularly easy to see in the main window when a **color palette** is selected and the **3D visualization** is activated.



If appropriate, **objects** in the scene **can be isolated first**. This speeds up the **correlation process** because only the areas where **objects are actually suspected are examined**. Isolation is also necessary when **multiple objects** in a scene need to be correlated simultaneously.

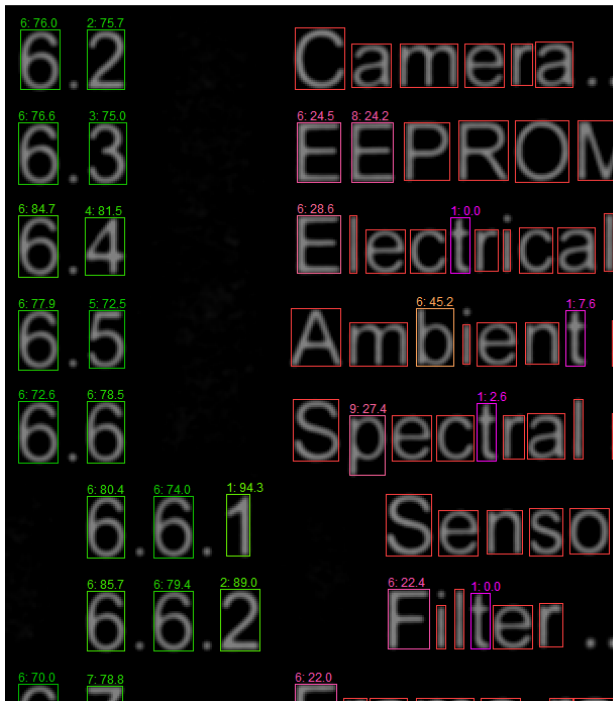
Simple binarization is used for pixel isolation. All pixels that are **lighter or darker** than the set **binarization threshold**

are classified as **objects**. Additional **size thresholds** allow for the differentiation of genuine objects from disturbances such as dirt or **debris**. Another setting is the **quality threshold**. It defines the value above which a correlation quality is rated as "good".

- Values **above** this threshold are displayed **in green**.
- Values **below** appear **yellow to pink**.
- **Very poor** results are marked **in red**.

0123
4567
89

0123456789
abcdefghijklmnop
rstuvwxy



A single **object image** can store **multiple individual objects**. For this to work, the objects in the image must be **clearly distinguishable from one another**, and a **text file with labels** must exist that has the **same base name** as the object image.

This technique can be used to implement a simple **text recognition**.

Isolate objects within the object image.

The following conditions apply in order for the software to recognize the individual objects:

- The objects must be surrounded by **border pixels**.
- **Object rows must be separated** by image rows that consist exclusively of border pixels.
- Within such a row, the objects must additionally be separated from each other by **columns of border pixels**.
- The **pixel in the top left corner** of the object image must always be a **border pixel**. The value of this pixel is used as a reference to identify border pixels.

Structure of the associated text file (labels):

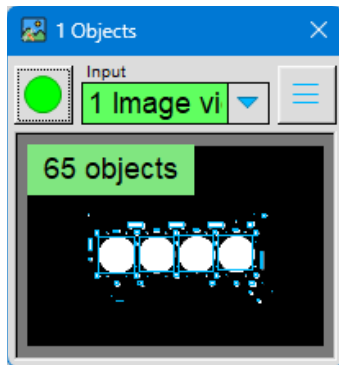
The text file must

- The **first line** contains the text "**Crcdf-Options**".
- Comment lines begin with "**///**".
- Each object should have a line with "**Label:**". The text to the right of this will later be used when displaying the correlation results.

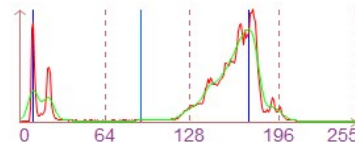
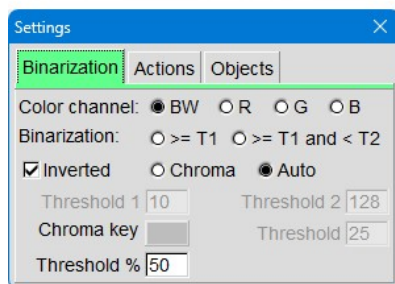
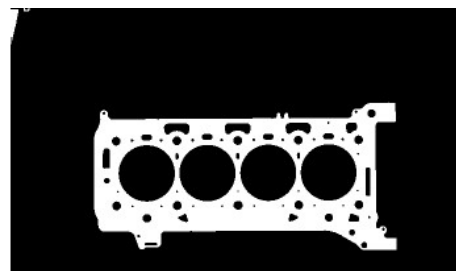
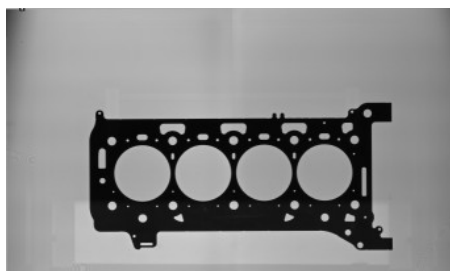
Important:

The **number of objects in the object image** must **exactly match** the **number of labels** in the text file. Only then can the objects be correctly assigned.

4.3.6 Objects



Separating objects from the **background** is a fundamental task in image processing. A prerequisite for this is that the **objects are clearly distinguishable from the background**. If this is not the case, a suitable background color or appropriate lighting can help.



Binarization converts the image into so-called **RLCs (Run Length Codes)**. RLCs are a compact representation of image lines and can be **processed much faster** than normal grayscale or color images.

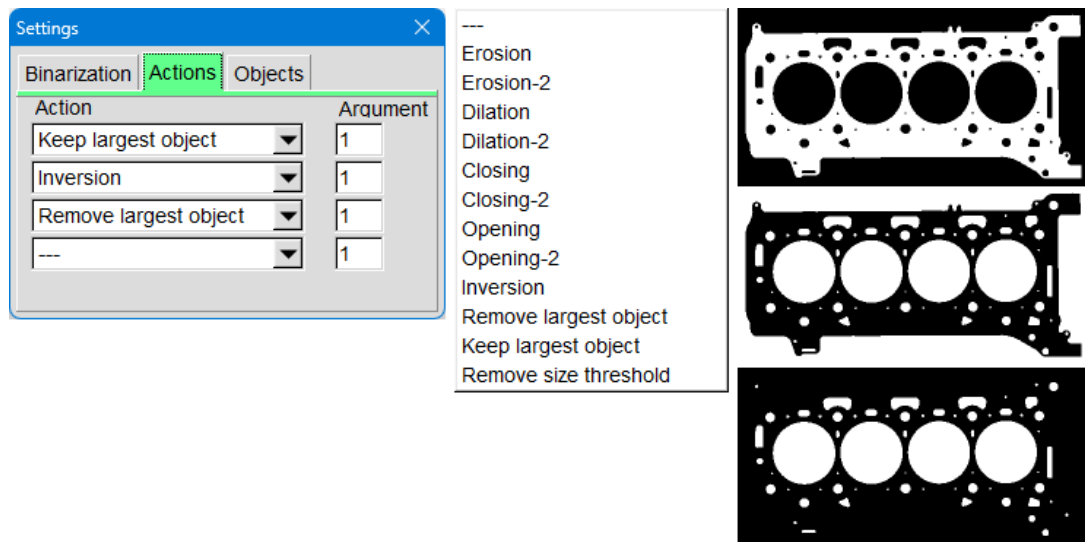
In addition to binarization with **one or two brightness thresholds**, a **color** can also be used as a criterion.

With the **"Auto" setting**, the binarization threshold is **automatically determined** via **histogram analysis**. For this to work, the image's **brightness histogram** must have **two clearly separated maxima**. The automatically calculated threshold is then a **percentage value between these two maxima**.

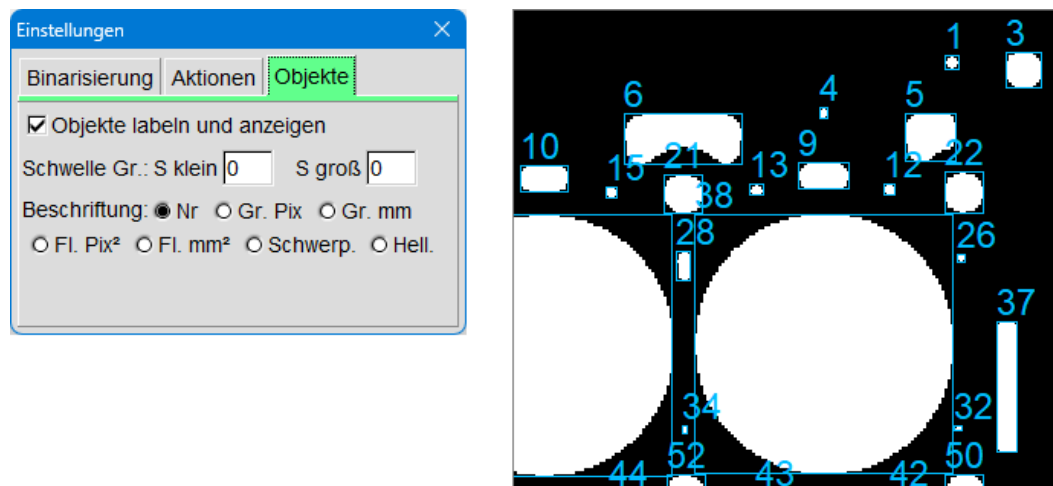
In the example above, you can see a **backlit engine block gasket** on the left and the binarization on the right.

The graphic shows:

- The brightness histogram is shown **in red**.
- **Green** is a smoothed version of this histogram.
- The identified maxima and the derived threshold are marked **in blue**.



Once the image has been binarized, various actions can be performed on the RLCs, such as **removing holes, noise pixels, or small unwanted objects**. The images above show the processing steps used to identify the holes in the seal.

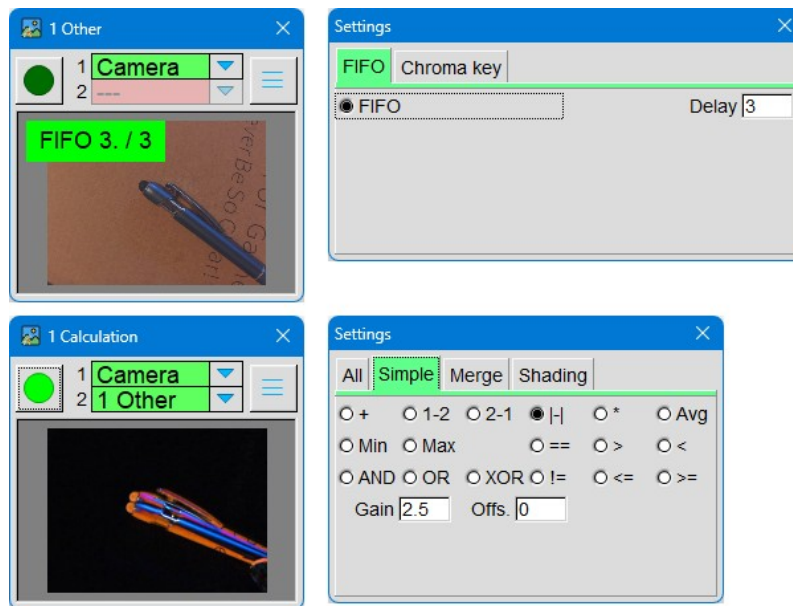


After the **RLCs** have been created, they are **labeled**. This involves grouping **contiguous image lines** into **objects**. Information about each object is also calculated, for example:

- Position and size of the **enclosing rectangle**
- Number of binarized pixels → **Area**
- The object's **center of gravity**
- or other features

4.3.7 Other

This tool window contains **various image processing functions** that cannot be assigned to any of the existing categories. It therefore serves as a **collection area for other processing steps** that do not fit into any of the defined groups.



FIFO (First In - First Out) outputs **incoming images with a time delay**. In the example shown, this simulates **motion detection**.

A camera focuses on a ballpoint pen. The current camera image is **compared with a time-delayed image** from the FIFO by **subtracting them from each other**.

If something moves in the image, this movement becomes **clearly visible in the difference image** because the pixel values differ between the current and the older image.



The **chroma key technique** separates the **foreground** from a **solid-colored background**. It's important that the **background color doesn't appear in the foreground**, otherwise the separation won't work reliably.

In addition to the background color, **threshold values for hue, saturation, and brightness must** be set. This allows you to precisely define which pixels are considered the background. Furthermore, various **output formats** and options for **optimizing the transition** between foreground and background are

available.

Note: When the **input image** is displayed in the main window and '**H AOI**' is selected as the measurement, a **measurement window** can be moved to a location on the background. The color measured there can be directly applied via the **context menu of the 'Color' button** using '**Get Color from Histogram**'.

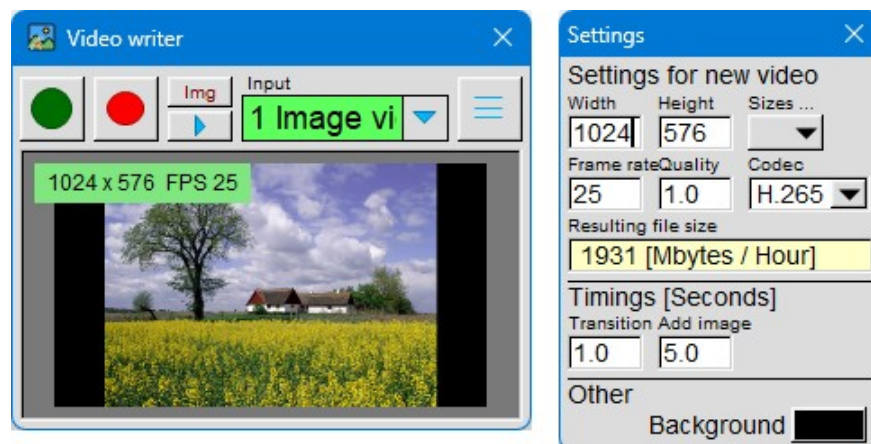


Here you can see a simple green screen setup.
If only small objects are in the foreground - for example, model cars - the entire setup can be made more compact.

4.4 Output

Output tools output an **input image**, for example, to a file or to a display. They always have an **input image**. Some generate an **output image**, but this cannot be processed further.

4.4.1 Creating a video



This tool can be **used to create videos**. Before recording starts, **all relevant** video settings should be configured and the **recording mode** selected.

The '**Image**' **recording mode** is intended for single images, e.g. from an image viewer. For each new image, frames are appended to the video for the set duration '**Add Image**'. A crossfade is created between the images for the '**Transition**' duration.

The '**Video**' **recording mode** is intended for **camera images**. Each **new camera image** is immediately **added** to the video as a **frame**.

4.5 Inspection

Industrial **image processing** uses cameras and software to **automatically** detect features, defects, or characteristics of products – usually directly on a production line. The goal is **to monitor** and **control manufacturing processes** or **to inspect finished products**.

The **inspection tools** shown demonstrate how such tests can be carried out. Typical tasks include:

- Take a reference image
- Measure or correct colors
- Position correction
- Measuring distances
- Compare image parts

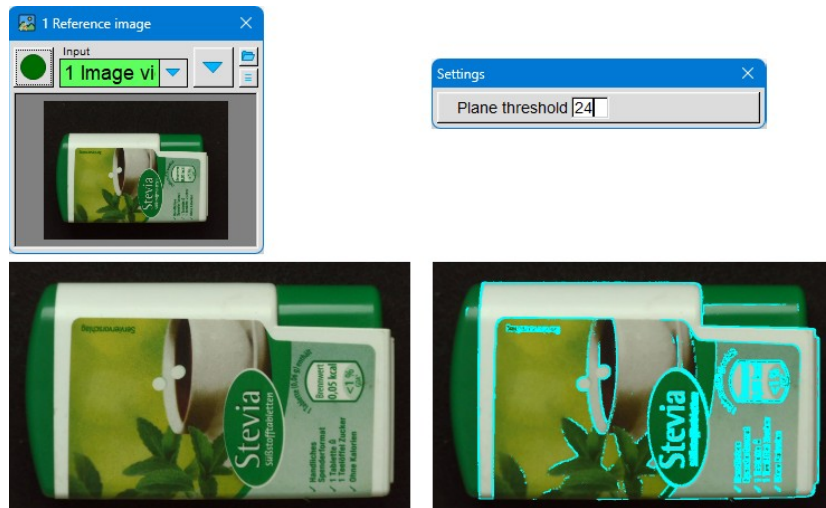
The **reference image** – for example, a camera image of a flawless object – serves as the basis. Other tools compare the **features of new images** with this reference image to **detect deviations**, **make errors visible**, or **compensate for changes in position**.



The images show the **setup used** and the **tool windows** that were used in the following descriptions.

Images of a small can were taken with the camera and saved in the application's '*Images\Demo\Inspection-1*' subdirectory. These examples can be easily opened with an image viewer, allowing you to follow the operation of the **inspection tools** step by step.

4.5.1 Reference image



In industrial image processing, a **reference image** serves as a *benchmark*. It shows a product **as it should appear** - flawless, correctly positioned, and properly lit. All subsequently captured images are compared to this reference image to reliably detect **deviations, errors, or changes**.

A reference image is the "**target version**" of a product against which all subsequent images are checked.

For the reference image to work reliably, several points must be correct:

- **Uniform lighting** No shadows, no reflections, no overexposure. The lighting must be reproducible later.
- **Clean, flawless object** The reference image must not contain any scratches, dust, fingerprints or production defects.
- **Stable position** The object must be positioned exactly as it will be later inspected - same position, same orientation.
- **Steady camera settings** Focus, exposure time, aperture, white balance must be set correctly and remain unchanged later.
- **High-contrast contours** edges, and shapes should be clearly recognizable so that image processing can reliably find them.
- **Stable environment** No changing light sources, no movement in the background, no vibrations.

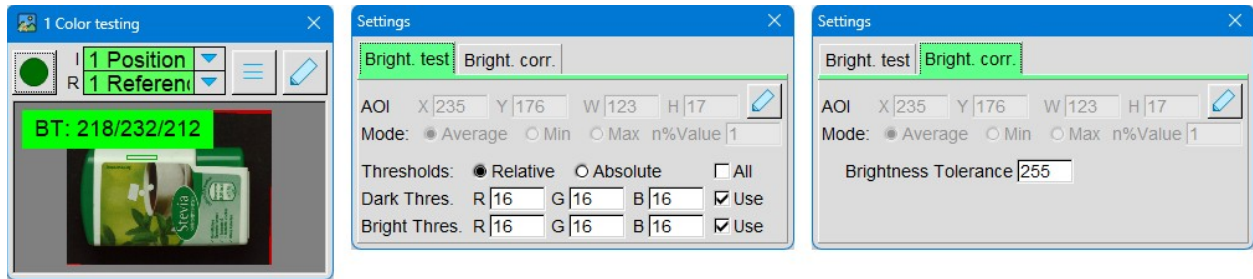
A **reference image** can be generated in two ways: Either an **input image** is used - for example, from a camera or an image viewer - or a **stored image** is loaded.

Next, the contour areas in the image are **marked**. This is done by examining the **neighboring pixels** of each pixel:

- If the **brightness difference** to all neighbors is **smaller** than the set "**Plane threshold**", the pixel is **low-pass filtered**.
- If the difference is **greater**, the pixel is classified as a **contour** and **remains unchanged**.
Additionally, a **contour bit** is set in the **alpha channel** of the reference image. This contour bit is used by the **image comparison tool** to improve comparisons in the contour area.

When the **parameter dialog is open**, the detected contour areas are visibly marked in the image.

4.5.2 Color Testing



In **learning mode**, the **reference image** is displayed. The **AOI** (Area of Interest) is overlaid on the image area whose color or brightness is to be measured.

The measurement is performed either as the **average of all pixels** within the AOI or as the **average of the darkest or brightest n% of pixels**.

This allows you to control whether the entire area is evaluated or only particularly bright or dark parts of the image.

Brightness test

In **evaluation mode**, the **brightness of the input image** is compared with the **brightness in the reference image**.

- If the difference is **within the error tolerance**, the image is considered **OK**. The measured value is highlighted **in green**.
- If the difference is **outside the tolerance range**, the image is **faulty**. The measured value is highlighted **in red**.

The brightness test is well suited for checking the **presence** of parts or for **color or brightness control** in production processes.

Brightness correction

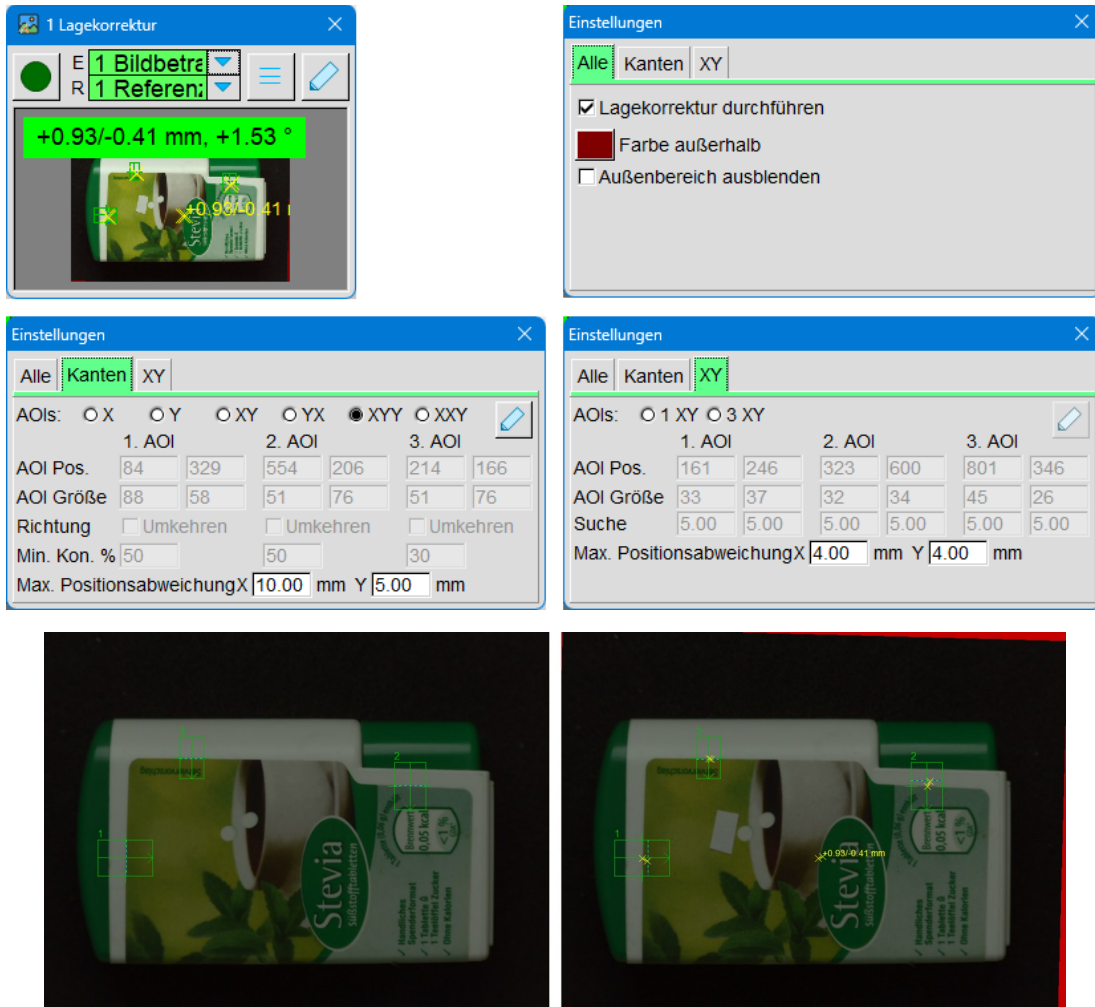
In **evaluation mode**, the **input image** is adjusted so that the **brightness in the AOI** corresponds to the value of the reference image.

- If the difference is **within tolerance**, the image is **OK** (green).
- If the difference is **greater than the tolerance**, the image is marked as **faulty** (red).

Brightness correction is particularly suitable for **compensating for changes in lighting**, e.g. due to aging of LEDs, slight fluctuations in light color or changes in light intensity.

Practical setup: Place a **bright reference area** in a corner of the camera image and position the AOI on it. As the LED lighting ages, both brightness and **color tone** change – both can be compensated for by the correction.

4.5.3 Position correction



This inspection tool corrects the **position of a part in the input image** so that the object in the **output image** has the **same position as in the reference image**.

- **Without position correction**, only the extent to which the object deviates from its position in the reference image is measured.
- **Position correction shifts** the **input image** accordingly so that the object is back in its target position.

Areas that **fall out of the image** due to this shift are either **colored in** or can be hidden using an **alpha mask**.

The **position-corrected output image** can then be further processed by other **inspection tools**.

Edge mode

In edge mode, **horizontal or vertical edges are** searched for. The **search direction is indicated by an arrow** in the AOI.

- In **learning mode**, the AOI is placed on a suitable edge. The **intensity of this edge** is saved.
- In **evaluation mode**, the **system searches** in the direction of the arrow for the **first edge** whose intensity corresponds **at least** to the set **minimum edge contrast**.

This allows the position of an object to be reliably determined based on a clear edge.

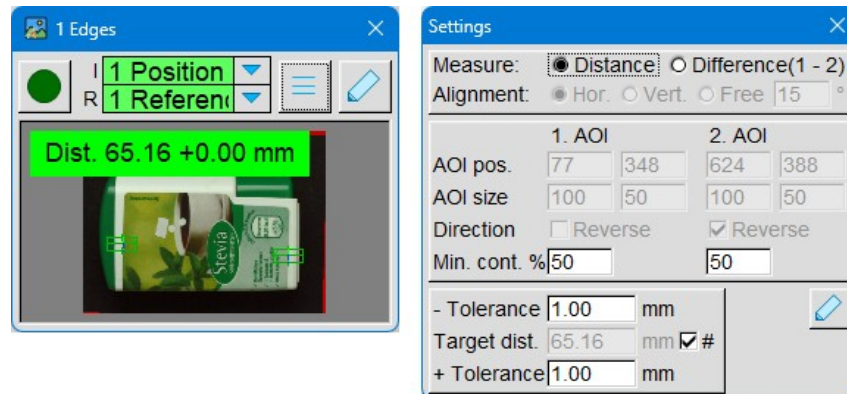
XY mode

In XY mode, the algorithm searches for areas of the image that **differ significantly** from their surroundings.

- In **learning mode**, the AOI is applied to such a prominent part of the image. The tool displays a **quality indicator**. the higher this value, the better suited the area is for position correction.
- The **dashed frame** around the AOI indicates the **search area**. This area should be **as small as possible** to ensure fast and stable position correction.

XY mode is particularly well-suited for objects with **distinctive structures** that consist of more than a single edge.

4.5.4 Edges



This tool measures **distances between edges**. To do this, the **AOIs are** placed on the desired edges. The **search direction** is indicated by an **arrow** within the AOI.

- In **learning mode**, the AOI is placed on a suitable edge. The **intensity of this edge** is saved.
- In **evaluation mode**, the **system searches** in the direction of the arrow for the **first edge** whose intensity corresponds **at least** to the set **minimum edge contrast**.

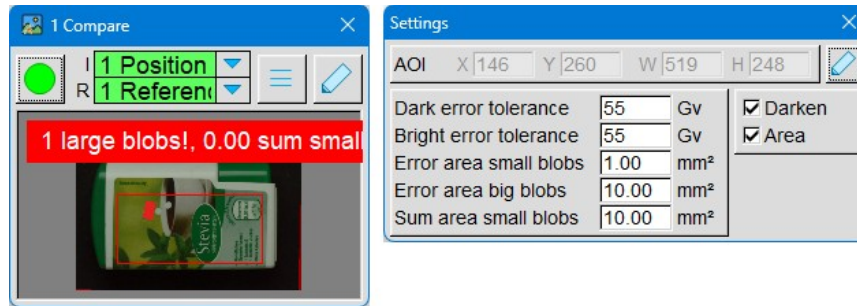
Difference

Here, the **difference between the two edge positions is** used as the measurement. The value can be **positive or negative**. This setting is particularly well-suited for **measuring offsets**, i.e., when you want to know whether an edge has been shifted to the left or right.

Distance

Here, the **absolute value of the difference is** used - always a **positive value**. This setting is well-suited for **size measurements** where only the **distance** between two edges is important.

4.5.5 Compare



This tool compares the **image content** between the **input image** and the **reference image**. The comparison only takes place **within the AOI**.

For each pixel, it is checked whether its **brightness** in the **input image** differs more from the reference image **than the set brightness tolerance**.

- Pixels with excessive deviation are considered **defective**.
- **Associated defective pixels** are grouped together as **blobs**.

The blobs are color-coded according to their size:

- **Subliminal blobs** → blue
- **Small blobs** → green
- **Large blobs** → red

The image comparison is considered **faulty** if:

- at least **one large blob** is present **or**
- the **sum of the areas of all small blobs** is above the set threshold.

To make the graphic more visible, the **output image** can be **darkened**.

Additionally, the **areas of smaller and larger blobs** can be displayed.

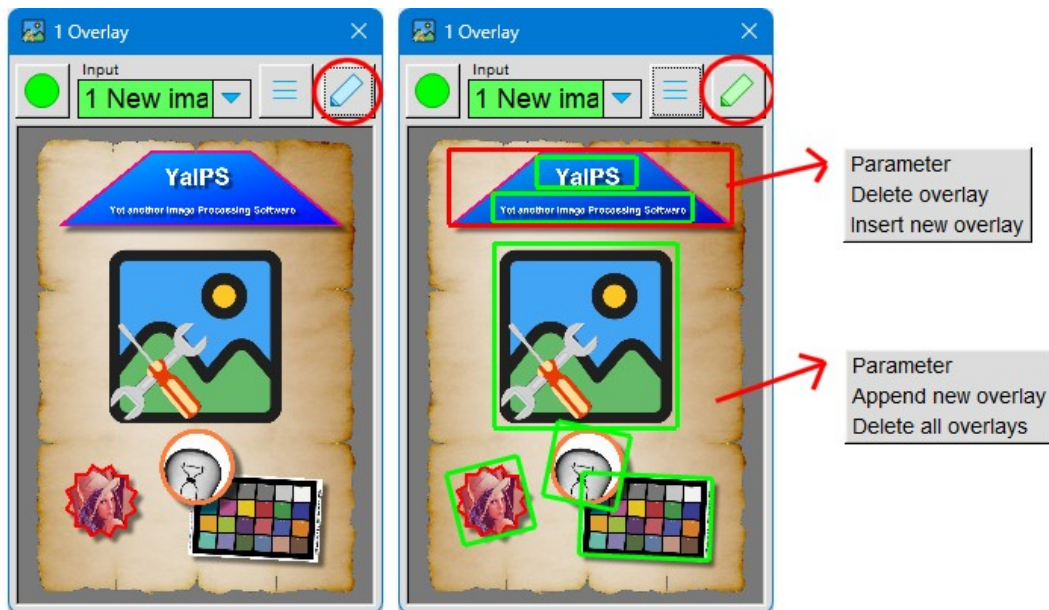
This tool is used for **surface inspection**, **print control**, and **other quality control** tasks.

Professional systems often use an additional **inspection mask**. A separate image is used to mark precisely the areas to be inspected. This allows the system to define **which image regions are relevant** and which can be ignored.

4.6 Other

Other tools contain functions that do not fit into any of the tool groups mentioned so far.

4.6.1 Overlay



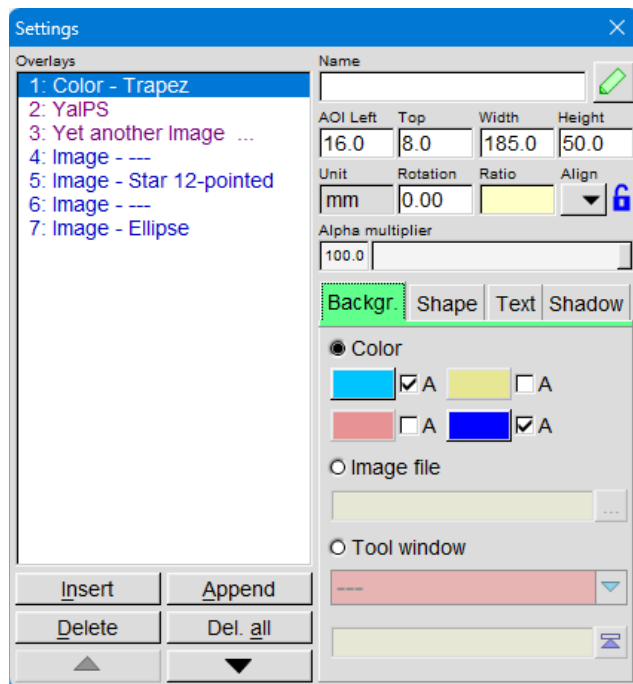
This tool allows you to overlay **other images, shapes, or text** onto an **image source**. The **output image** always has the same size as the image source.

If "New Image" is used as the image source, a **poster with a defined size and resolution** – such as DIN A4 at 150 dpi – can be generated, along with the calibration. If a **camera** is used as the image source, **text** or **logos** can be superimposed on the camera's **live image**.

Up to 16 overlays can be defined.

Operation in learning mode and closed parameter dialog:

- **Left mouse button:** Move or resize overlay.
- **Right mouse button over an overlay:**
 - Open parameter dialog of this overlay
 - Delete overlay
 - Insert new overlay
- **Right mouse button over the background:**
 - Open parameter dialog
 - Add a new overlay at the end
 - Delete all overlays
- **Mouse wheel:**
 - Holding **Ctrl** down: Rotate overlay by 1°
 - Holding **Shift** down: Rotate overlay by 15°
- **Dragging an image file onto an overlay:** If the overlay has an image as its background, this image will be **replaced**.
- **Dragging the image file onto the background:** A new overlay will be inserted at the end of the list.



The **left half** of the parameter dialog contains a **list of all overlays**. They are drawn **in the order of the list** - the **last overlay** is always on top and therefore **visible**, while earlier overlays can be covered by later ones.

You can edit the currently selected overlay using the buttons at the **bottom**:

- **Move overlay**
- **Delete overlay**
- **Insert a new overlay in front**
- **Append new overlay afterwards**

If the **Ctrl-key** is pressed when creating new overlays, they will be created as a **copy of the selected one**.

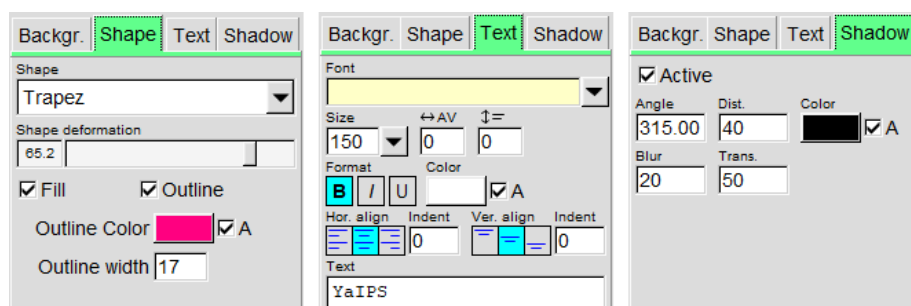
In the upper right part of the dialog, you can make further adjustments to the overlay:

- Change **position** and **size**
- a **name** (this will then appear as text in the list)
- **Rotate** the superposition
- the overlay **at the image edges**
- **Lock** the aspect ratio to prevent the shape from being distorted when scaling
- Set transparency

In the **lower right part** of the parameter dialog, there are several **tabs** that can be used to edit different properties of an overlay.

Background: In this tab, you select what is used as the background of the overlay: **color**, **image file**, or **tool window**.

For image file and tool window, the **size is also** displayed. Using the adjacent button, you can **directly apply this image size as the size of the overlay**.



Shape: Here you can select one of the available **shapes**. The shape will be **placed over the background**. Optionally, a **colored outline can be added**.

Text: In this tab, you can **enter text** that will be displayed in the overlay. Besides **font** and **font size**, numerous other settings are available, such as **alignment**, **line spacing**, **color**, and **transparency**.

Shadows: Each overlay can have a **shadow**. The following properties can be set:

- **Direction of the shadow**
- **Color**
- **Blur**
- **transparency**

This allows overlays to be visually highlighted or rendered in a more three-dimensional way.